Rockchip Developer Guide DisplayPort

ID: RK-KF-YF-466

Release Version: V1.2.1

Release Date: 2025-06-18

Security Level: □Top-Secret □Secret □Internal ■Public

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document mainly introduces the usage and debugging methods of the DP interface on the Rockchip platform.

Product Version

Chipset	Kernel Version
RK3576	LINUX Kernel 6.1
RK3588	LINUX Kernel 5.10/6.1

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Author	Date	Change Description
V1.0.0	Zhang Yubing	2022-05-26	Initial version
V1.1.0	Zhang Yubing	2024-01-15	Added functional configuration description
V1.2.0	Zhang Yubing	2024-03-25	Added RK3576 configuration description
V1.2.1	Huang Tao	2025-01-21	Corrected spelling errors

Contents

1. Introduction to Rockchip Platform DisplayPort

1.1 Functional Features

The functional parameters of the RK3576 and RK3588 DP interfaces on the Rockchip platform are as follows:

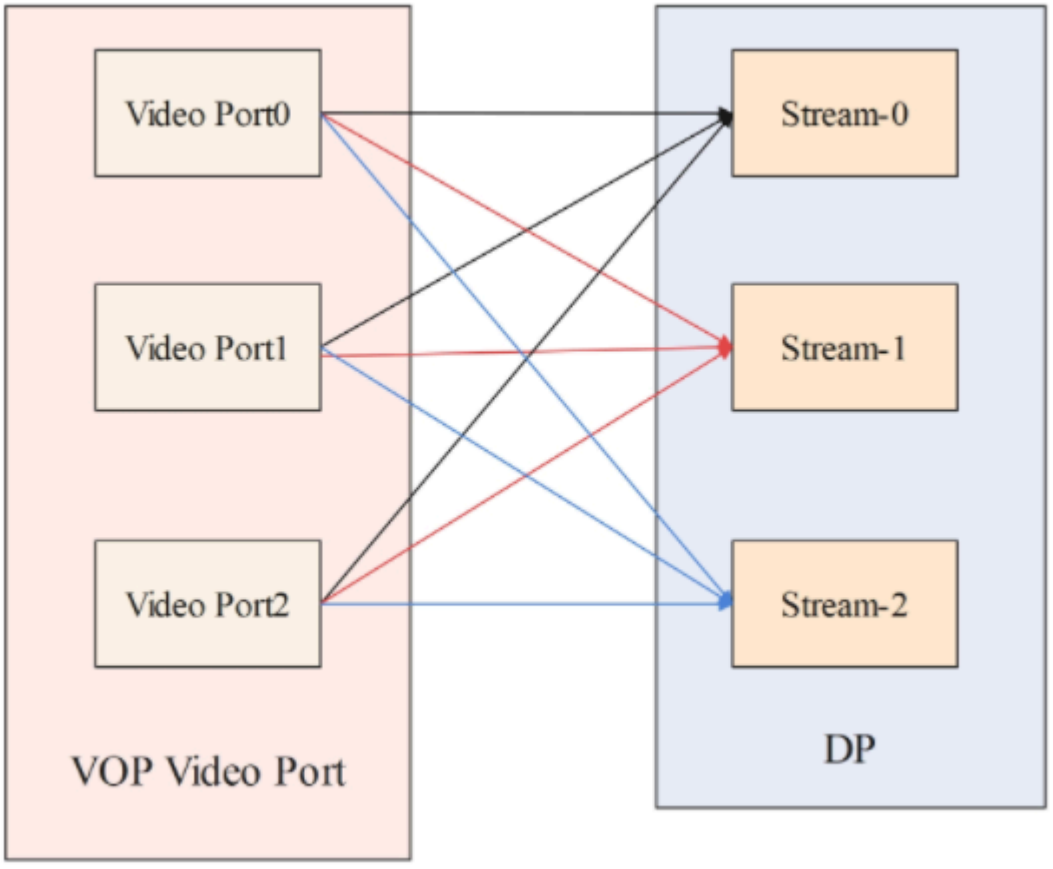
Feature	RK3576	RK3588
Version	1.4a	1.4a
SST	Supported	Supported
MST	Supported	Not supported
DSC	Not supported	Not supported
Max resolution	4K@120Hz	8K@30Hz
Main-Link lanes	1/2/4 lanes	1/2/4 lanes
Main-Link rate	8.1/5.4/2.7/1.62 Gbps/lane	8.1/5.4/2.7/1.62 Gbps/lane
AUX_CH	1 M	1 M
Color Format	RGB/YUV444/YUV422/YUV420	RGB/YUV444/YUV422/YUV420
Color Depth	8/10 bit (6bit only for RGB)	8/10 bit (6bit only for RGB)
Display Split Mode	Supported	Supported
HDCP	HDCP2.2/HDCP1.3	HDCP2.2/HDCP1.3
Type-C support	DP Alternate Mode	DP Alternate Mode
I2S	Supported	Supported
SPDIF	Supported	Supported
HDR	Supported	Supported

The RK3576 has only one physical DP interface, but it can accept 3 display data streams internally in MST mode (to distinguish the physical interface, Stream-0, Stream-1, and Stream-2 are used). The maximum output capability of each stream is as follows:

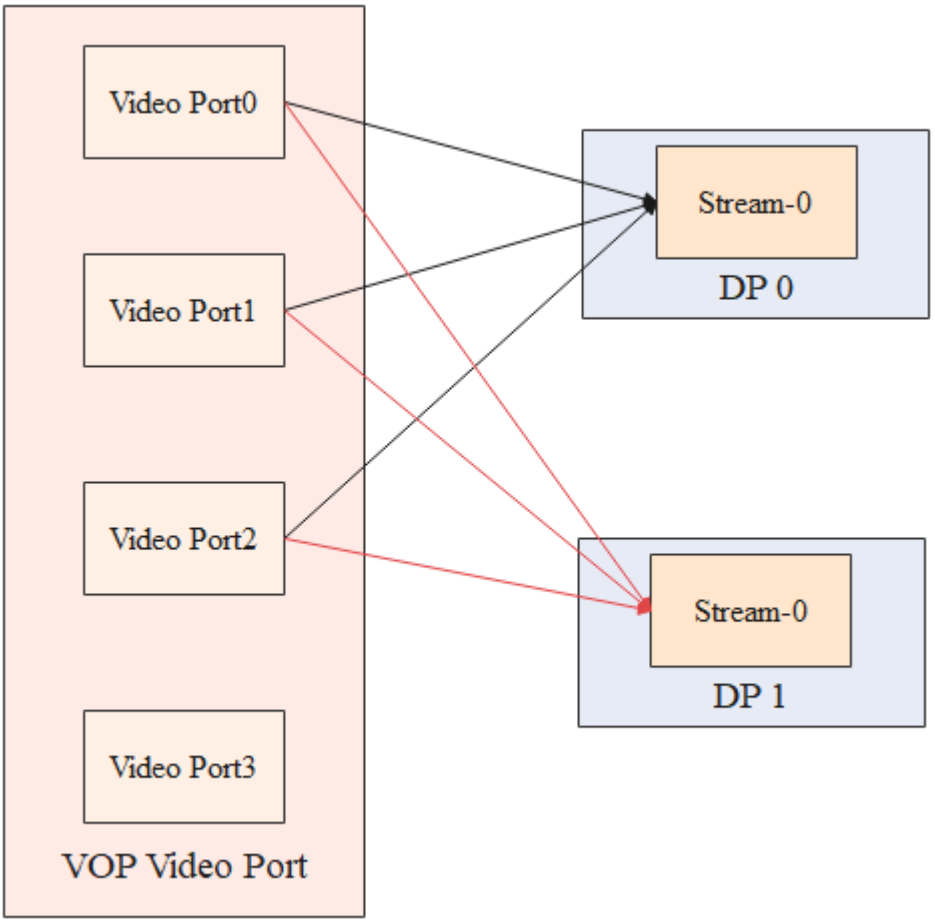
DP Stream Channel	max width	max height	max pixel clock
Stream-0	4096	2160	1188MHz
Stream-1	2560	1440	300MHz
Stream-2	1920	1080	150MHz

1.2 Connection Relationship between DP and VOP

The VOP of RK3576 has 3 Video Ports and one DP controller. In MST mode, the DP controller can receive up to 3 display data streams from the VOP. Stream-0/1/2 can all receive display data from Video Port 0/1/2. When operating in SST mode, only Stream-0 in the DP controller can be used. When operating in MST mode, Stream-0/1/2 can all be used.



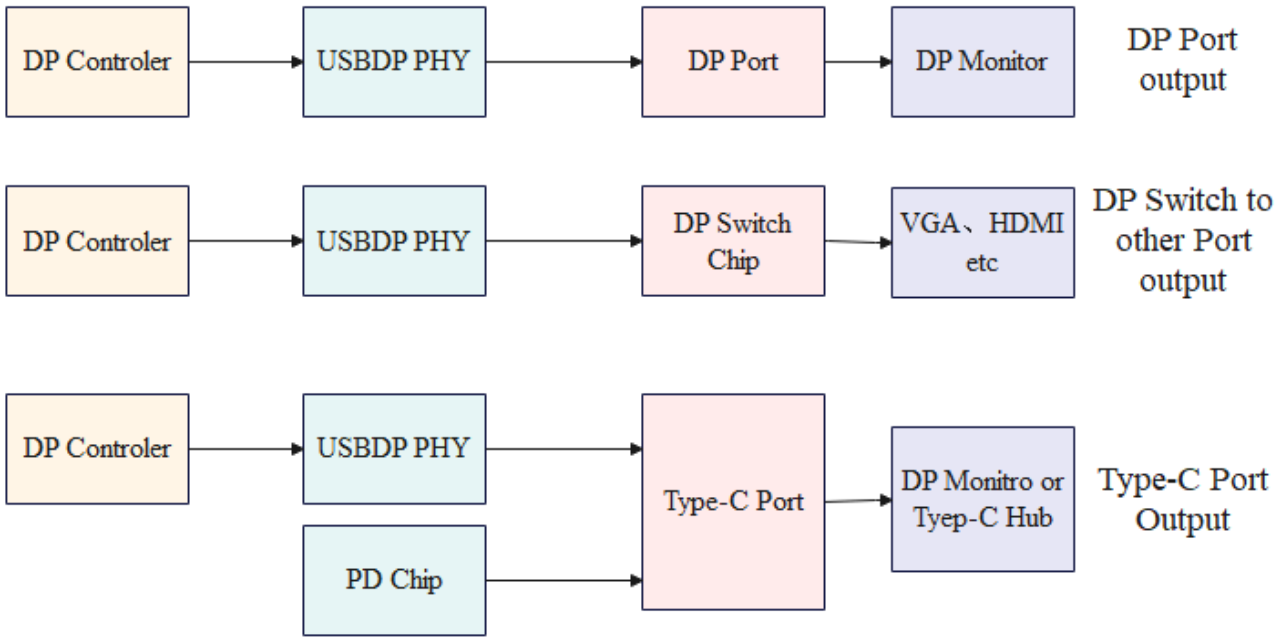
The VOP of RK3588 has 4 Video Ports and two DP controllers, but only Video Port 0/1/2 can output to DP0/1, as shown in the figure below.



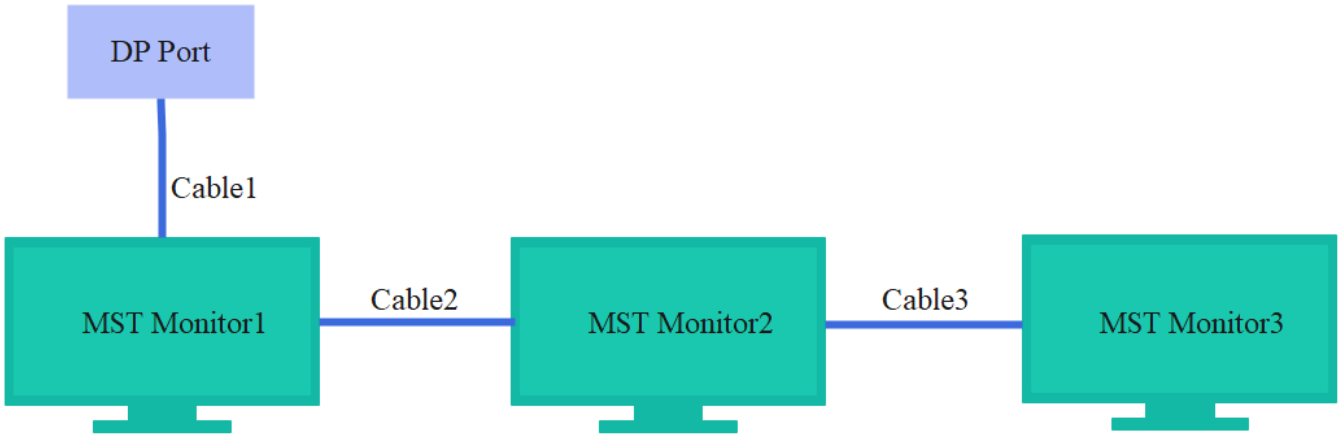
Since the 2 DP interfaces of RK3588 do not support MST mode and can only receive one display data stream internally (Stream-0), the default Video Port output is connected to the Stream-0 of the DP interface for platforms that do not support MST.

1.3 DP Output

Depending on the application scenario, different DP output methods can be designed: Type-C interface output, standard DP interface output, and output through other conversion chips.

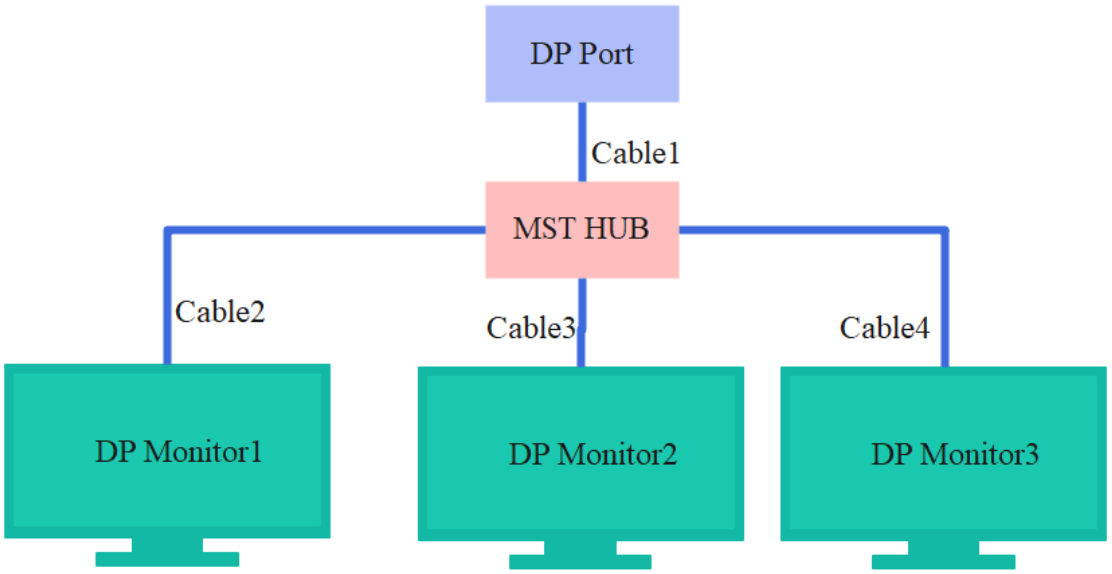


In MST mode, RK3576 can connect up to 3 monitors, which can be daisy-chained through MST monitors, as follows:



For monitors connected through a daisy chain, only the last monitor can be an SST monitor, while the others need to be MST monitors.

Another method is to connect through an MST HUB, as follows:



When connecting through an MST HUB, the DP monitors can be either SST monitors or MST monitors.

1.4 Code Path

U-Boot driver code:

```
drivers/video/drm/dw-dp.c
drivers/phy/phy-rockchip-usbdp.c
```

Kernel driver code:

```
drivers/gpu/drm/rockchip/dw-dp.c
drivers/phy/rockchip/phy-rockchip-usbdp.c
```

RK3576 reference DTS configuration:

```
arch/arm64/boot/dts/rockchip/rk3576-evb1.dtsi
arch/arm64/boot/dts/rockchip/rk3576-test2.dtsi
```

RK3588 reference DTS configuration:

```
arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi
arch/arm64/boot/dts/rockchip/rk3588-evb2-lp4.dtsi
arch/arm64/boot/dts/rockchip/rk3588-evb3-lp5.dtsi
arch/arm64/boot/dts/rockchip/rk3588-nvr-demo.dtsi
```

1.5 Driver Loading

Determine whether the driver is loaded by the following log:

```
RK3576:
 [1.991964] rockchip-drm display-subsystem: bound 27e40000.dp (ops 0xffffffc0094a1570) // DP
driver loaded done
RK3588:
 [2.472282] rockchip-drm display-subsystem: bound fde50000.dp (ops dw_dp_component_ops) //
DP0 driver loaded done
 [2.472319] rockchip-drm display-subsystem: bound fde60000.dp (ops dw_dp_component_ops) //
DP1 driver loaded done
```

2. Functional Configuration

For the DP interface, there are differences in the basic configuration of the DTS nodes between platforms that support MST and those that do not.

For platforms that do not support MST, such as RK3588, one DP controller only supports one DP output. It is only necessary to define one ports sub-node to describe the display path that this DP can support. The DP node description is as follows:

```
dp0: dp@fde50000 {
    compatible = "rockchip,rk3588-dp";
    ...

    ports {
        #address-cells = <1>;
        #size-cells = <0>;

        port@0 {
            reg = <0>;
            #address-cells = <1>;
            #size-cells = <0>;

            dp0_in_vp0: endpoint@0 {
                reg = <0>;
                remote-endpoint = <&vp0_out_dp0>;
                status = "disabled";
            };

            dp0_in_vp1: endpoint@1 {
                reg = <1>;
                remote-endpoint = <&vp1_out_dp0>;
                status = "disabled";
            };

            dp0_in_vp2: endpoint@2 {
```

```
                reg = <2>;
                remote-endpoint = <&vp2_out_dp0>;
                status = "disabled";
```

For platforms that support MST, such as RK3576, one DP controller needs to support 3 display data streams. A single ports node cannot describe the display paths for multiple DP channels, so multiple sub-nodes are required for the configuration, as follows:

```
dp: dp@27e40000 {
    compatible = "rockchip,rk3576-dp";
    ...
    dp0: dp0 {
        status = "disabled";

        ports {
            #address-cells = <1>;
            #size-cells = <0>;

            port@0 {
                reg = <0>;
                #address-cells = <1>;
                #size-cells = <0>;

                dp0_in_vp0: endpoint@0 {
                    reg = <0>;
                    remote-endpoint = <&vp0_out_dp0>;
                    status = "disabled";
                };

                dp0_in_vp1: endpoint@1 {
                    reg = <1>;
                    remote-endpoint = <&vp1_out_dp0>;
                    status = "disabled";
                };

                dp0_in_vp2: endpoint@2 {
                    reg = <2>;
                    remote-endpoint = <&vp2_out_dp0>;
                    status = "disabled";
                };
            };
        };
    };

    dp1: dp1 {
        status = "disabled";

        ports {
            #address-cells = <1>;
            #size-cells = <0>;
```

```
            port@0 {
                reg = <0>;
                #address-cells = <1>;
                #size-cells = <0>;

                dp1_in_vp0: endpoint@0 {
                    reg = <0>;
                    remote-endpoint = <&vp0_out_dp1>;
                    status = "disabled";
                };

                dp1_in_vp1: endpoint@1 {
                    reg = <1>;
                    remote-endpoint = <&vp1_out_dp1>;
                    status = "disabled";
                };

                dp1_in_vp2: endpoint@2 {
                    reg = <2>;
                    remote-endpoint = <&vp2_out_dp1>;
                    status = "disabled";
                };
            };
        };
    };

    dp2: dp2 {
        status = "disabled";

        ports {
            #address-cells = <1>;
            #size-cells = <0>;
            port@0 {
                reg = <0>;
                #address-cells = <1>;
                #size-cells = <0>;

                dp2_in_vp0: endpoint@0 {
                    reg = <0>;
                    remote-endpoint = <&vp0_out_dp2>;
                    status = "disabled";
                };

                dp2_in_vp1: endpoint@1 {
                    reg = <1>;
                    remote-endpoint = <&vp1_out_dp2>;
                    status = "disabled";
                };

                dp2_in_vp2: endpoint@2 {
                    reg = <2>;
                    remote-endpoint = <&vp2_out_dp2>;
                    status = "disabled";
```

```
                };
            };
        };
    };
};
```

The above dp0/1/2 sub-nodes describe the display paths that Stream-0/1/2 in the DP controller can support, respectively.

Compared to the DTS configuration, an additional DP channel sub-node is present on platforms that support MST.

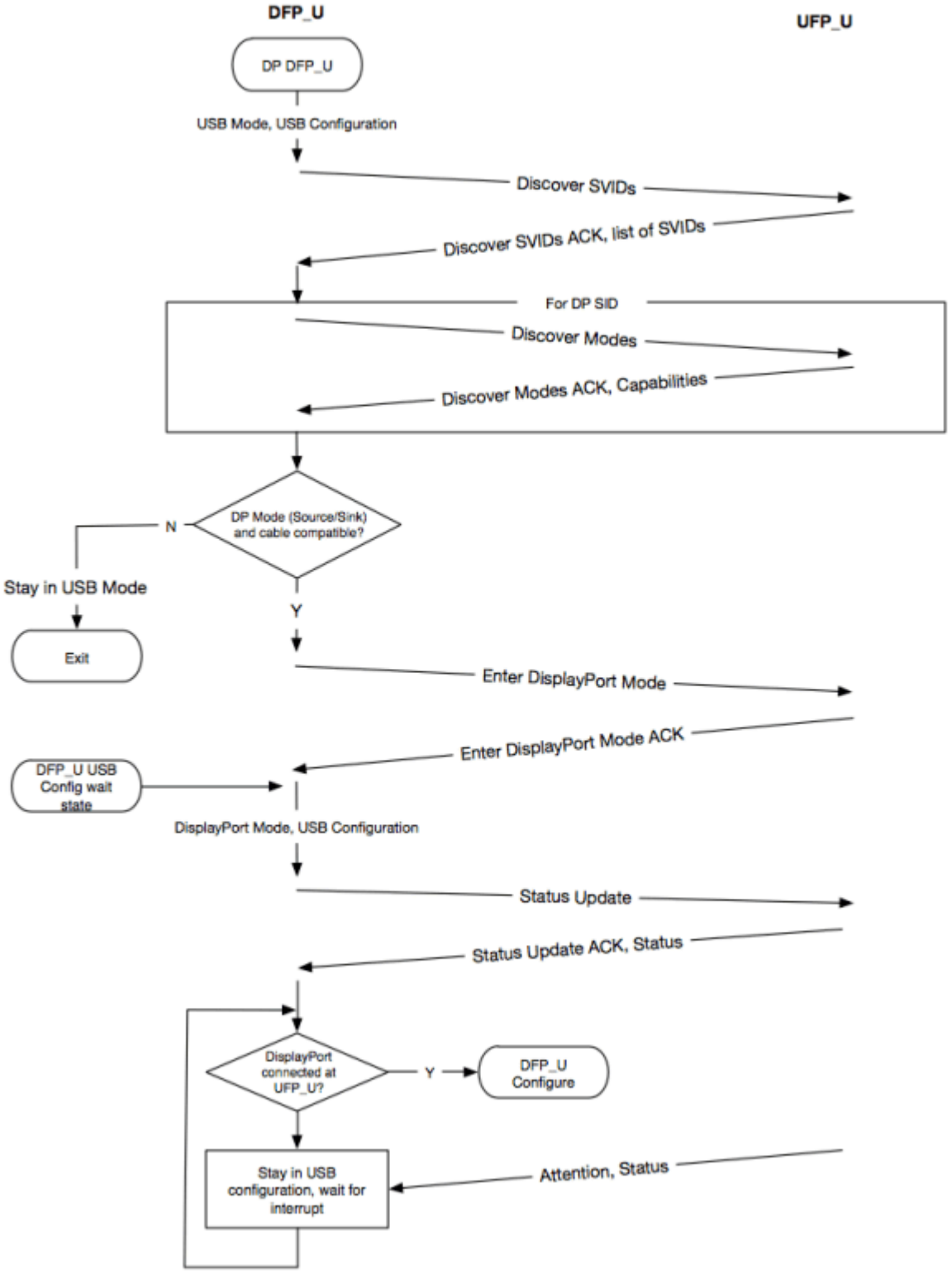
2.1 Enabling DP

DP and USB3.0 share the PHY, and the PHY lane configuration varies depending on the interface, with 2 modes: Type-C mode and legacy mode.

2.1.1 DP Alt Mode (Type-C)

According to the DisplayPort Alt Mode protocol, communication with the monitor is conducted through the PD (Power Delivery) state machine to map lanes and transmit HPD information. The process of entering DP Mode via the PD protocol and transmitting HPD information through the attention command is mainly shown in the figure below.

Figure 5-2 DFP_U DisplayPort Alternate Mode Discovery and Entry



For platforms that do not support MST, such as RK3588, the configuration is as follows:

```
&dp0 {
    status = "okay";
};

&dp0_in_vp2 {
    status = "okay";
};
```

In the above configuration, the DP0 interface is enabled, and DP0 is bound to Video Port 2 of the VOP. This is just a reference configuration. In actual use, you can enable DP0 or DP1 and bind DP0 or DP1 to the desired Video Port (0/1/2) according to your specific needs.

For platforms that support MST, such as RK3576, the configuration is as follows:

```
&dp {
    status = "okay";
};

&dp0 {
    status = "okay";
};

&dp0_in_vp2 {
    status = "okay";
};
```

It can be seen that for platforms that support MST, it is necessary to enable the DP device node, the DP Stream channel to be opened, and the Video Port on the VOP that this channel is bound to. In the above configuration, the DP interface's Stream-0 is enabled, and Stream-0 is bound to Video Port 2 of the VOP.

It should be noted that for platforms that support MST, since Stream-0 must be used in SST mode, the dp0 node must be enabled. dp1 and dp2 are configured according to usage.

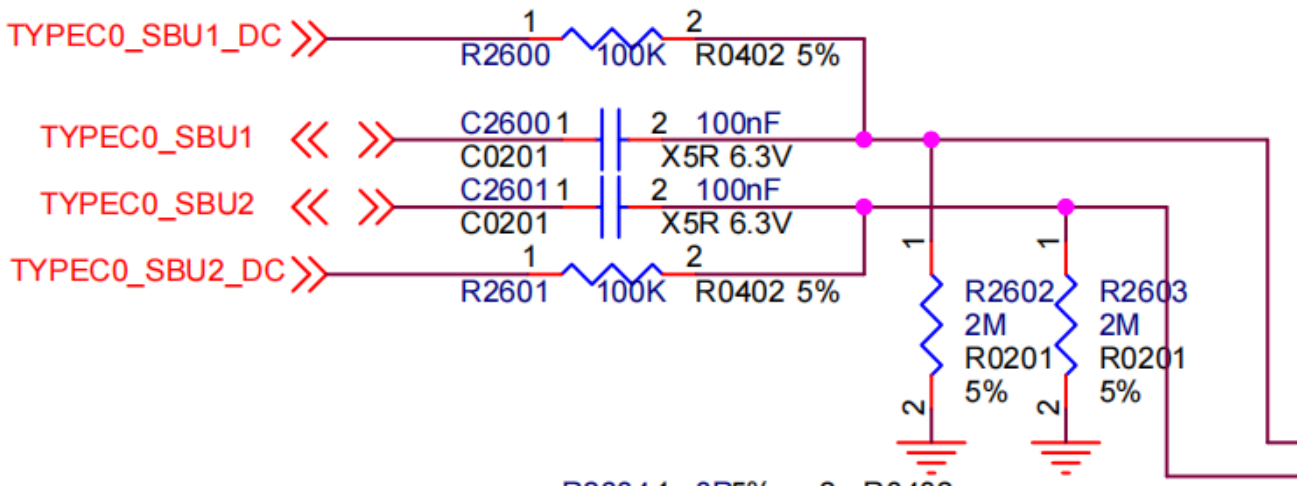
The PHY configuration is as follows, with no differences between platforms that support MST and those that do not. Refer to the following RK3588 usbdp phy0 configuration:

```
&usbdp_phy0 {
    status = "okay";
    orientation-switch;
    /* DP related config */
    svid = <0xff01>;
    sbu1-dc-gpios = <&gpio4 RK_PA6 GPIO_ACTIVE_HIGH>;
    sbu2-dc-gpios = <&gpio4 RK_PA7 GPIO_ACTIVE_HIGH>;
    /* DP related config */

    port {
        #address-cells = <1>;
        #size-cells = <0>;
        usbdp_phy0_orientation_switch: endpoint@0 {
            reg = <0>;
            remote-endpoint = <&usbc0_orien_sw>;
        };
```

```
        /* DP related config */
        usbdp_phy0_dp_altmode_mux: endpoint@1 {
            reg = <1>;
            remote-endpoint = <&dp_altmode_mux>;
        };
        /* DP related config */
    };
};
```

`sbu1-dc-gpios` and `sbu2-dc-gpios`:

The SBU1 and SBU2 pins of Type-C are multiplexed with the DP AUX_CH. When Type-C is inserted in normal, AUX_CH_P is multiplexed with SBU1, and AUX_CH_N is multiplexed with SUB2. When Type-C is inserted in reverse, AUX_CH_P is multiplexed with SBU2, and AUX_CH_N is multiplexed with SBU1. According to the DP protocol requirements, AUX_CH_P needs to be configured as a pull-down state, and AUX_CH_N needs to be configured as a pull-up state. The multiplexing configuration of AUX_CH_N and AUX_CH_P is different depending on the insertion state of Type-C (normal or reverse insertion). In the RK solution, 2 GPIOs are used to control the pull-up and pull-down states of SBU1 and SBU2, respectively, that is, the sbu1-dc-gpios and sbu2-dc-gpios in the dts. Therefore, when configuring the PHY, it is necessary to configure `sbu1-dc-gpios` and `sbu2-dc-gpios` (when actually configuring these 2 GPIOs, refer to the hardware design schematic, for example, TYPEC0_SBU1_DC and TYPEC0_SBU2_DC in the figure below), and the PHY driver will adjust the GPIO output levels according to the current Type-C insertion state.



`svid`:

For DP, the fixed value is 0xff01.

The Type-C interface requires configuration of lane and HPD states through Type-C CC detection and PD negotiation, so it is also necessary to configure the PD chip:

```
&i2c2 {
    status = "okay";

    usbc0: fusb302@22 {
        compatible = "fcs,fusb302";
        reg = <0x22>;
        interrupt-parent = <&gpio3>;
        interrupts = <RK_PB4 IRQ_TYPE_LEVEL_LOW>;
```

```
        pinctrl-names = "default";
        pinctrl-0 = <&usbc0_int>;
        vbus-supply = <&vbus5v0_typec>;
        status = "okay";

        ports {
            #address-cells = <1>;
            #size-cells = <0>;

            port@0 {
                reg = <0>;
                usbc0_role_sw: endpoint@0 {
                    remote-endpoint = <&dwc3_0_role_switch>;
                };
            };
        };

        usb_con: connector {
            compatible = "usb-c-connector";
            label = "USB-C";
            data-role = "dual";
            power-role = "dual";
            try-power-role = "sink";
            op-sink-microwatt = <1000000>;
            sink-pdos =
                <PDO_FIXED(5000, 1000, PDO_FIXED_USB_COMM)>;
            source-pdos =
                <PDO_FIXED(5000, 3000, PDO_FIXED_USB_COMM)>;

            /* DP related config */
            altmodes {
                #address-cells = <1>;
                #size-cells = <0>;

                altmode@0 {
                    reg = <0>;
                    svid = <0xff01>;
                    vdo = <0xffffffff>;
                };
            };
            /* DP related config */

            ports {
                #address-cells = <1>;
                #size-cells = <0>;

                port@0 {
                    reg = <0>;
                    usbc0_orien_sw: endpoint {
                        remote-endpoint = <&usbdp_phy0_orientation_switch>;
                    };
                };
```

```
                /* DP related config */
                port@1 {
                    reg = <1>;
                    dp_altmode_mux: endpoint {
                        remote-endpoint = <&usbdp_phy0_dp_altmode_mux>;
                    };
                };
                /* DP related config */
            };
        };
    };
};
```

In the `altmode@0` node, `svid` is fixed at 0xff01, and `vdo` is fixed at 0xffffffff.

Note: The currently supported PD chips are fusb302 and hub311. The driver for fusb302 is located at `/drivers/usb/typec/tcpm/fusb302.c`, and the driver for hub311 is located at `/drivers/usb/typec/tcpm/tcpci_husb311.c`.

2.1.2 DP Legacy Mode

For non-Type-C interface outputs, whether it is a DP interface or output through other conversion chips, the configuration process is essentially the same and requires HPD Pin configuration. When actually allocating IO pins, you can use the dedicated DP_HPD pin, which is configured according to IOMUX, or you can use a regular GPIO for detection.

For platforms that do not support MST, such as RK3588, when using the DP_HPD Pin, the configuration is as follows:

```
&dp1 {
    pinctrl-0 = <&dp1m2_pins>;
    pinctrl-names = "default";
    status = "okay";
};

&dp1_in_vp2 {
    status = "okay";
};
```

When using a regular GPIO for HPD detection, the configuration is as follows:

```
&dp1 {
    pinctrl-names = "default";
    pinctrl-0 = <&dp1_hpd>;
    hpd-gpios = <&gpio1 RK_PB5 GPIO_ACTIVE_HIGH>;
    status = "okay";
};

&dp1_in_vp2 {
    status = "okay";
```

```
};

&pinctrl {
    dp {
        dp1_hpd: dp1-hpd {
            rockchip,pins = <1 RK_PB5 RK_FUNC_GPIO &pcfg_pull_down>;
        };
    };
};
```

For platforms that support MST, such as RK3576, when using the DP_HPD Pin, the configuration is as follows:

```
&dp {
    pinctrl-0 = <&dp1m2_pins>;
    pinctrl-names = "default";
    status = "okay";
};

&dp0 {
    status = "okay";
};

&dp0_in_vp2 {
    status = "okay";
};
```

When using a regular GPIO for HPD detection, the configuration is as follows:

```
&dp {
    pinctrl-names = "default";
    pinctrl-0 = <&dp_hpd>;
    hpd-gpios = <&gpio1 RK_PB5 GPIO_ACTIVE_HIGH>;
    status = "okay";
};

&dp0 {
    status = "okay";
};

&dp0_in_vp2 {
    status = "okay";
};

&pinctrl {
    dp {
        dp_hpd: dp-hpd {
            rockchip,pins = <1 RK_PB5 RK_FUNC_GPIO &pcfg_pull_down>;
        };
    };
};
```

In the above configurations for both MST and non-MST platforms, the HPD configuration belongs to the entire DP interface configuration and is configured under the device node.

DP and USB 3.0 share the PHY. When DP is output through a non-Type-C interface, it is necessary to specify which lane is configured for DP use and the corresponding lane number, which is defined in the DTS. For the DP PHY lane configuration, it can be set to either a 2-lane mode or a 4-lane mode.

The physical numbering of the PHY lane and the pin relationship are as follows:

Pin Name	SSRX1	SSTX1	SSRX2	SSTX2
Phy Lane	0	1	2	3

For a DP configuration with 4 lanes, the dtsi configuration attribute is as follows:

```
rockchip,dp-lane-mux = <x x x x>;
```

For a DP configuration with 2 lanes, the dtsi configuration attribute is as follows:

```
rockchip,dp-lane-mux = <x x>;
```

Here, the index represents the DP logic lane, and the value represents the PHY lane.

Regardless of whether it is a 2-lane or 4-lane configuration, 2 options, OPTION1 or OPTION2, are generally used in hardware design.

Pin Name	Type-C Function	DPx4Lane Function		USB30 HOST+DPx2Lane Function	
		OPTION1	OPTION2	OPTION1	OPTION2
TYPEC0_SBU1/DP0_AUXP TYPEC0_SBU2/DP0_AUXN	TYPEC0_SBU1 TYPEC0_SBU2	DP0_AUXP DP0_AUXN	DP0_AUXP DP0_AUXN	DP0_AUXP DP0_AUXN	DP0_AUXP DP0_AUXN
TYPEC0_SSRX1P/DP0_TX0P TYPEC0_SSRX1N/DP0_TX0N	TYPEC0_SSRX1P TYPEC0_SSRX1N	DP0_TX0P DP0_TX0N	DP0_TX2P DP0_TX2N	TYPEC0_SSRX1P TYPEC0_SSRX1N	DP0_TX0P DP0_TX0N
TYPEC0_SSTX1P/DP0_TX1P TYPEC0_SSTX1N/DP0_TX1N	TYPEC0_SSTX1P TYPEC0_SSTX1N	DP0_TX1P DP0_TX1N	DP0_TX3P DP0_TX3N	TYPEC0_SSTX1P TYPEC0_SSTX1N	DP0_TX1P DP0_TX1N
TYPEC0_SSRX2P/DP0_TX2P TYPEC0_SSRX2N/DP0_TX2N	TYPEC0_SSRX2P TYPEC0_SSRX2N	DP0_TX2P DP0_TX2N	DP0_TX0P DP0_TX0N	DP0_TX2P DP0_TX2N	TYPEC0_SSRX2P TYPEC0_SSRX2N
TYPEC0_SSTX2P/DP0_TX3P TYPEC0_SSTX2N/DP0_TX3N	TYPEC0_SSTX2P TYPEC0_SSTX2N	DP0_TX3P DP0_TX3N	DP0_TX1P DP0_TX1N	DP0_TX3P DP0_TX3N	TYPEC0_SSTX2P TYPEC0_SSTX2N
TYPEC0_USB20_OTG_DP TYPEC0_USB20_OTG_DM	TYPEC0_USB20_OTG_DP TYPEC0_USB20_OTG_DM			TYPEC0_USB20_OTG_DP TYPEC0_USB20_OTG_DM	TYPEC0_USB20_OTG_DP TYPEC0_USB20_OTG_DM

For the OPTION1 mapping relationship of DP 4 lanes:

DP lane	Phy lane	Pin Name
0	0	SSRX1
1	1	SSTX1
2	2	SSRX2
3	3	SSTX2

Where DP lane represents the sequence number of the DP lane.

The dts configuration is as follows:

```
&usbdp_phy1 {
    rockchip,dp-lane-mux = <0 1 2 3>;
    status = "okay";
};
```

For the OPTION2 mapping relationship of DP 4 lanes:

DP lane	Phy lane	Pin Name
0	2	SSRX2
1	3	SSTX2
2	0	SSRX1
3	1	SSTX1

Where DP lane represents the sequence number of the DP lane.

The dts configuration is as follows:

```
&usbdp_phy1 {
    rockchip,dp-lane-mux = <2 3 0 1>;
    status = "okay";
};
```

For the OPTION1 mapping relationship of DP 2 lanes:

DP lane	Phy lane	Pin Name
0	2	SSRX2
1	3	SSTX2

The DP 2-lane configuration is as follows:

```
&usbdp_phy1 {
    rockchip,dp-lane-mux = <2 3>;
    status = "okay";
};
```

For the OPTION2 mapping relationship of DP 2 lanes:

DP lane	Phy lane	Pin Name
0	0	SSRX1
1	1	SSTX1

The DP 2-lane configuration is as follows:

```
&usbdp_phy1 {
    rockchip,dp-lane-mux = <0 1>;
    status = "okay";
};
```

2.2 DP Connecting to Panel Peripheral

When using the DP interface to connect to an eDP Panel, unique features of eDP such as PSR, Multi-SST, and ALPM are not supported. The panel configuration can be referenced as follows and adjusted according to the actual hardware design:

For platforms that do not support MST, such as RK3588, the configuration is as follows:

```
/ {
    ...
    panel-edp {
        compatible = "simple-panel";
        backlight = <&backlight>;
        power-supply = <&vcc3v3_lcd_edp>;
        prepare-delay-ms = <120>;
        enable-delay-ms = <120>;
        unprepare-delay-ms = <120>;
        disable-delay-ms = <120>;
        width-mm = <120>;
        height-mm = <160>;

        panel-timing {
            clock-frequency = <200000000>;
            hactive = <1536>;
            vactive = <2048>;
            hfront-porch = <12>;
            hsync-len = <16>;
            hback-porch = <48>;
            vfront-porch = <8>;
            vsync-len = <4>;
            vback-porch = <8>;
            hsync-active = <0>;
            vsync-active = <0>;
            de-active = <0>;
            pixelclk-active = <0>;
        };

        port {
            panel_in_edp: endpoint {
                remote-endpoint = <&dp0_out>;
            };
        };
    };
    ...
};
```

```
&dp0 {
    force-hpd;
    status = "okay";
};

&dp0_in_vp2 {
    status = "okay";
};

&dp0_out {
    remote-endpoint = <&panel_in_edp>;
};

&usbdp_phy0 {
    rockchip,dp-lane-mux = <0 1 2 3>;
    status = "okay";
};
```

For platforms that support MST, such as RK3576, the corresponding configuration is as follows:

```
/ {
    ...
    panel-edp {
        compatible = "simple-panel";
        backlight = <&backlight>;
        power-supply = <&vcc3v3_lcd_edp>;
        prepare-delay-ms = <120>;
        enable-delay-ms = <120>;
        unprepare-delay-ms = <120>;
        disable-delay-ms = <120>;
        width-mm = <120>;
        height-mm = <160>;

        panel-timing {
            clock-frequency = <200000000>;
            hactive = <1536>;
            vactive = <2048>;
            hfront-porch = <12>;
            hsync-len = <16>;
            hback-porch = <48>;
            vfront-porch = <8>;
            vsync-len = <4>;
            vback-porch = <8>;
            hsync-active = <0>;
            vsync-active = <0>;
            de-active = <0>;
            pixelclk-active = <0>;
        };

        port {
            panel_in_edp: endpoint {
                remote-endpoint = <&dp0_out_panel>;
```

```
            };
        };
    };
    ...
};

&dp {
    force-hpd;
    status = "okay";
};

&dp0 {
    status = "okay";

    ports {
        port@1 {
            reg = <1>;

            dp0_out_panel: endpoint {
                remote-endpoint = <&panel_in_edp>;
            };
        };
    };
};

&dp0_in_vp2 {
    status = "okay";
};

&usbdp_phy {
    rockchip,dp-lane-mux = <0 1 2 3>;
    status = "okay";
};
```

In the above configuration, the `force-hpd` attribute describes the HPD of the entire DP interface and should be placed under the device node. The display path configuration is related to the specific DP display path, so for MST platforms, the specific display path sub-node needs to be modified.

For platforms that support MST, currently, when connecting to a panel, it can only operate in SST mode, so the display path can only use Stream-0, corresponding to the dp0 node.

In the above configuration, after the force-hpd attribute is configured in the dp0 node, the driver assumes that the eDP panel is always connected by default. This attribute is not mandatory and should be confirmed whether to configure the force-hpd attribute based on whether the specific panel has an HPD pin, whether HPD is pulled high, and whether there are timing requirements for AUX access. If the panel requires AUX access to occur after HPD is pulled high, the force-hpd attribute should not be configured, otherwise, it may lead to AUX access failure before HPD is pulled high. If it is still necessary to configure the HPD pin, refer to the HPD configuration in 1.2.1 DP Legacy Mode.

The panel-timing configuration supports the current timing. If the eDP panel does not have EDID, or the timing read from EDID is inaccurate, it is necessary to configure the panel-timing node. Otherwise, it can be omitted and directly obtained through reading EDID.

The power-up and power-down timing and backlight are configured according to the specific screen and hardware design.

2.3 DP Boot Logo

After configuring the boot logo, if a DP monitor is inserted before booting, the logo can be displayed during the U-Boot stage. Otherwise, the image displayed by the application can only be seen after the system is started. The configuration to add DP boot logo support is as follows:

```
&route_dp0 {
    status = "okay";
    connect = <&vp2_out_dp0>;
};
```

It should be noted that the connect attribute here configures DP to bind to VOP Port 2 during the U-Boot stage, so the dtsi configuration should allow DP to bind to VOP Port 2:

```
&dp0_in_vp2 {
    status = "okay";
};
```

Note:

1. Currently, DP boot logos for Type-C interfaces are not supported;
2. For platforms that support MST, boot logos are only supported in SST mode.

2.4 DP Connector-Split Mode

The DP connector-split mode is shown in the figure below, where an image is evenly split into 2 halves and transmitted to the monitor through DP0/DP1 interfaces respectively. In the figure below, DP0 is the left half of the screen, and DP1 is the right half of the screen.

The configuration is as follows:

```
&dp0 {
    split-mode;
    status = "okay";
};

&dp0_in_vp2 {
    status = "okay";
};

&dp1 {
    status = "okay";
};
```

Add the `split-mode` attribute to the DP node that serves as the left half of the screen and bind it to the Video Port to be output. In the above configuration, DP0 is the left screen and DP1 is the right screen. In Split Mode, the 2 DP interfaces act as one connector. The connector is only in a connected state when both DP0 and DP1 are connected, and display output will only start then. If either DP interface is disconnected, the connector is considered disconnected and no display output will occur. In this mode, the timing of the output from the two DP interfaces is the same, and it is recommended to use 2 identical monitors.

In user space, through modetest or by viewing the state node of dri (`cat /sys/kernel/debug/dri/0/state`), only one DP connector will be seen.

If you want to display the U-boot logo in split mode, for example, with DP0 as the left half of the screen, the reference configuration to be added is as follows:

```
&route_dp0 {
    split-mode;
    status = "okay";
    connect = <&vp2_out_dp0>;
};

&route_dp1 {
    status = "disabled";
}
```

Note: RK3576 has only one interface and does not support this type of connector-split mode. The split-mode function supported by RK3576 will be supplemented later. If there is a need for the split-mode function on RK3576, please contact Rockchip.

2.5 HDR

HDR functionality is supported by default in SST mode, and no driver configuration is required. HDR functionality under MST is currently not supported.

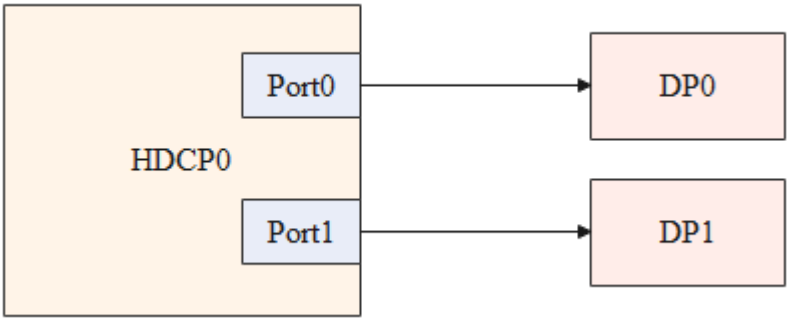
2.6 HDCP

The DP driver implements HDCP functionality based on the DRM framework. To use HDCP, users need to call the DRM interface in user space.

HDCP1.3 is supported by default in the DP driver and does not require configuration. For HDCP Key burning, refer to the "Rockchip_RK3588_Developer_Guide_HDCP_CN" document.

HDCP2.2 has a separate HDCP2 controller to manage HDCP authentication. Enabling the HDCP2 controller requires DTS configuration.

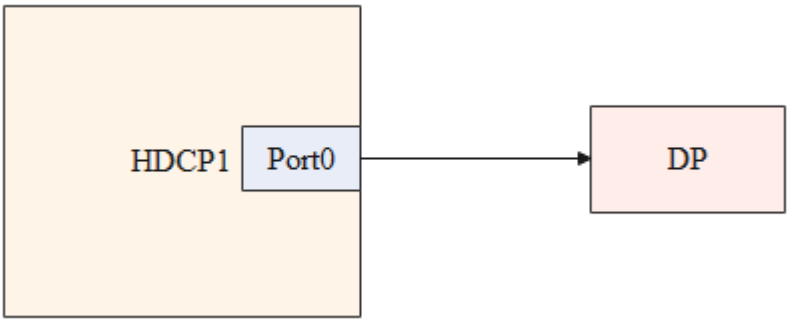
On RK3588, DP0/DP1 is connected to HDCP0, as shown in the figure below:



To enable DP HDCP2.2 functionality, the following node needs to be enabled:

```
&hdcp0 {
    status = "okay";
};
```

On RK3576, DP is connected to HDCP1, as shown in the figure below:



To enable DP HDCP2.2 functionality, the following node needs to be enabled:

```
&hdcp1 {
    status = "okay";
};
```

In addition to driver configuration for HDCP2.2, it is also necessary to enable the HDCP2.2 application in user space and generate the HDCP controller firmware. Refer to the "Rockchip_RK3588_Developer_Guide_HDCP_EN" document.

3. Common DEBUG Methods

3.1 Viewing Connector Status

In the `/sys/class/drm` directory, you can see the various cards registered by the driver. In the following displayed content, `card0-DP-1` and `card0-DP-2` are DP display devices:

```
rk3588_s:/ # ls /sys/class/drm/
card0       card0-DP-2   card0-HDMI-A-1  card0-Writeback-1  renderD128  version
card0-DP-1  card0-DSI-1  card0-HDMI-A-2  card1              renderD129
```

Taking `card0-DP-1` as an example, its directory contains the following content:

```
rk3588_s:/ # ls /sys/class/drm/card0-DP-1/
device  dpms  edid  enabled  modes  power  status  subsystem  uevent
```

Check the enabled status:

```
rk3588_s:/ # cat /sys/class/drm/card0-DP-1/enabled
disabled
```

Check the connection status:

```
rk3588_s:/ # cat /sys/class/drm/card0-DP-1/status
disconnected
```

List of supported resolutions for the device:

```
rk3588_s:/ # cat /sys/class/drm/card0-DP-1/modes
1440x900
1280x1024
1280x1024
1280x960
1152x864
1024x768
1024x768
832x624
800x600
800x600
640x480
640x480
720x400
```

EDID of the device, which can be saved using the following command:

```
rk3588_s:/ # cat /sys/class/drm/card0-DP-1/edid > /data/edid.bin
```

3.2 Forcing DP Enable/Disable

```
# Force DP to be disabled
rk3588_s:/ # echo off > /sys/class/drm/card0-DP-1/status
# Force DP to be enabled
rk3588_s:/ # echo on > /sys/class/drm/card0-DP-1/status
# Restore hotplug detection
rk3588_s:/ # echo detect > /sys/class/drm/card0-DP-1/status
```

3.3 DPCD Read/Write

DPCD is read and written through AUX_CH. The implementation of the read/write node is located in:

```
/drivers/gpu/drm/drm_dp_aux_dev.c
```

Before using this feature, ensure that the relevant compilation options have been configured:

```
CONFIG_DRM_DP_AUX_CHARDEV=y
```

Reading DPCD is as follows:

```
#if the value after is the aux node, when 2 DP interfaces are registered, there will be
/dev/drm_dp_aux0 and /dev/drm_dp_aux1
#skip value is the starting DPCD register address
#count value is the number of DPCD registers to be read
dd if=/dev/drm_dp_aux0 bs=1 skip=$((0x00200)) count=2 status=none | od -tx1
#The following is the content of the two DPCD registers starting at address 0x00200
rk3588_s:/ # dd if=/dev/drm_dp_aux1 bs=1 skip=$((0x00200)) count=2 status=none | od -tx1
0000000 01 00
0000002
```

Writing to DPCD registers:

```
#echo is the value to be written, the following are two 16-bit hexadecimal values, 0x0a and
0x80 respectively
#of is the aux node, when two DP interfaces are registered, there will be /dev/drm_dp_aux0
and /dev/drm_dp_aux1
#seek is the starting DPCD register address
#count value is the number of DPCD registers to be written
#The following command writes the two values 0x0a and 0x80 to the two DPCD registers
starting at address 0x100
echo -e -n "\x0a\x80" | dd of=/dev/drm_dp_aux0 bs=1 seek=$((0x100)) count=2 status=none
```

3.4 Type-C Interface Debugging

The HPD detection part of the Type-C interface is completed by the PD chip, and the software process of this part is mainly completed by the TCPM framework. The TCPM detection log can be obtained in the following way:

```
rk3588_s:/ # ls -l /sys/kernel/debug/usb/
total 0
-r--r--r--  1 root root 0 1970-01-01 00:00 devices
drwxr-xr-x 18 root root 0 1970-01-01 00:00 fc000000.usb
drwxr-xr-x  2 root root 0 1970-01-01 00:00 fc400000.usb
-r--r--r--  1 root root 0 1970-01-01 00:00 fusb302-2-0022
drwxr-xr-x  4 root root 0 1970-01-01 00:00 ohci
-r--r--r--  1 root root 0 1970-01-01 00:00 tcpm-2-0022
drwxr-xr-x  2 root root 0 2021-01-01 12:00 uvcvideo
drwxr-xr-x  3 root root 0 1970-01-01 00:00 xhci
```

In the `/sys/kernel/debug/usb/` directory, you can see `fusb302-2-0022` and `tcpm-2-0022`, where `fusb302-2-0022` is the node for the PD chip, and `tcpm-2-0022` is the node for the TCPM framework. The command to obtain the TCPM framework log is as follows:

```
cat /sys/kernel/debug/usb/tcpm-2-0022
```

Note: In `tcpm-2-0022`, the middle `2` represents the corresponding i2c bus, and the last `0022` represents the i2c address of the PD chip.

The command to obtain the PD chip log is as follows:

```
cat /sys/kernel/debug/usb/fusb302-2-0022
```

Note: In `fusb302-2-0022`, the middle `2` represents the corresponding i2c bus, and the last `0022` represents the i2c address of the PD chip. The above node corresponds to the fusb302 chip. The node name for different chips will be different.

In addition to the log, there is other information that can be obtained under the Type-C node. The path to the Type-C node is as follows:

```
console:/ # ls /sys/class/typec
port0  port0-partner
```

`port0` represents the Type-C interface on the SoC side, and `port0-partner` represents the node directory of the connected device after connecting through the Type-C interface.

Information about the orientation of the Type-C connection:

```
cat /sys/class/typec/port0/orientation
reverse
```

Under `port0-partner`, there may be multiple directories. For the directory corresponding to DP Alt Mode, it will have a `displayport` subdirectory, and the value of `svid` will be `0xff01`.

```
ls -l /sys/class/typec/port0-partner/port0-partner.0/
total 0
-r--r--r-- 1 root root 4096 2022-04-14 14:50 active
-r--r--r-- 1 root root 4096 2022-04-14 14:50 description
drwxr-xr-x 2 root root    0 2022-04-14 14:50 displayport
lrwxrwxrwx 1 root root    0 2022-04-14 14:50 driver ->
../../../../../../../../../bus/typec/drivers/typec_displayport
-r--r--r-- 1 root root 4096 2022-04-14 14:50 mode
drwxr-xr-x 2 root root    0 2022-04-14 14:50 mode1
lrwxrwxrwx 1 root root    0 2022-04-14 14:50 port -> ../../port0.0
drwxr-xr-x 2 root root    0 2022-04-14 14:50 power
lrwxrwxrwx 1 root root    0 2022-04-14 14:50 subsystem ->
../../../../../../../../../bus/typec
-r--r--r-- 1 root root 4096 2022-04-14 14:50 svid
-rw-r--r-- 1 root root 4096 2022-04-14 14:50 uevent
-r--r--r-- 1 root root 4096 2022-04-14 14:50 vdo
```

```
cat /sys/class/typec/port0-partner/port0-partner.0/svid
ff01
```

Obtain the current pin assignment information:

```
cat /sys/class/typec/port0-partner/port0-partner.0/displayport/pin_assignment
C [D]
#The currently connected device supports C assignment and D assignment, and the current
configuration is D assignment
```

Note: The above describes the information acquisition of PD chips using the TCPM framework. If the PD chip used in combination is not based on the TCPM framework, please confirm the relevant information with the PD chip vendor.

3.5 Viewing DP Registers

RK3588 DP related registers:

```
#dp0 controller
cat /sys/kernel/debug/regmap/fde50000.dp/registers
#usbdp phy0
cmn_reg0000 - cmn_reg015D:
io -4 -r -l 1400 0xfed88000
trsv_reg0200 - trsv_reg03C3:
io -4 -r -l 1808 0xfed88800
trsv_reg0400 - trsv_reg0435:
io -4 -r -l 212 0xfed89000
trsv_reg0600  - trsv_reg07C3:
io -4 -r -l 1808 0xfed89800
trsv_reg0800  - trsv_reg0835:
io -4 -r -l 212 0xfed8A000
#dp1 controller
cat /sys/kernel/debug/regmap/fde60000.dp/registers
#usbdp phy1
cmn_reg0000 - cmn_reg015D:
io -4 -r -l 1400 0xfed98000
trsv_reg0200 - trsv_reg03C3:
io -4 -r -l 1808 0xfed98800
trsv_reg0400 - trsv_reg0435:
io -4 -r -l 212 0xfed99000
trsv_reg0600  - trsv_reg07C3:
io -4 -r -l 1808 0xfed99800
trsv_reg0800  - trsv_reg0835:
io -4 -r -l 212 0xfed9A000
# vo0_grf
cat /sys/kernel/debug/regmap/dummy-syscon@fd5a6000/registers
```

RK3576 DP related registers:

```
#dp controller
cat /sys/kernel/debug/regmap/27e40000.dp/registers
#usbdp phy
cmn_reg0000 - cmn_reg015D:
io -4 -r -l 1400 0x2b018000
trsv_reg0200 - trsv_reg03C3:
io -4 -r -l 1808 0x2b018800
trsv_reg0400 - trsv_reg0435:
io -4 -r -l 212 0x2b019000
trsv_reg0600  - trsv_reg07C3:
io -4 -r -l 1808 0x2b019800
```

```
trsv_reg0800  - trsv_reg0835:
io -4 -r -l 212 0x2b01a000
# vo1_grf
cat /sys/kernel/debug/regmap/dummy-syscon@0x0000000026036000/registers
```

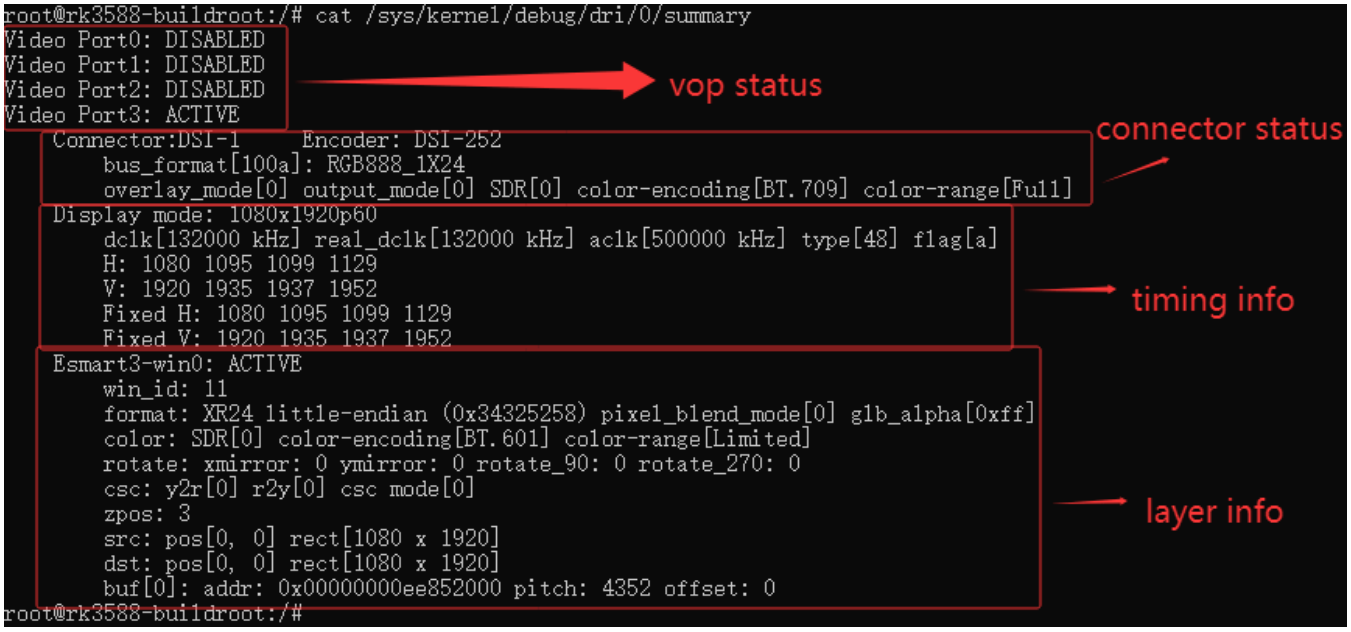
Note: The PHY registers can only be dumped when a DP monitor is connected and displaying properly.

3.6 Viewing VOP Status

The status of the VOP can be queried using the following command:

```
cat /sys/kernel/debug/dri/0/summary
```

The obtained VOP status is shown in the figure below:



Video Portx: Indicates the current status of the Video Port

Connector: The output interface currently connected to the Video Port

Display mode: The current output timing of the Video Port

Clusterx-winx(Esmartx-winx): Layer information

In Kernel 6.1 and above versions, the obtained information is as follows:

```
root@linaro-alip:/# cat /sys/kernel/debug/dri/0/summary
Video Port0: ACTIVE
    Connector:DP-2      Encoder: DP-MST 0
        bus_format[100a]: RGB888_1X24
        overlay_mode[0] output_mode[f] SDR[0] color-encoding[BT.709] color-range[Limited]
    Display mode: 1280x720p60
        clk[74250] real_clk[74250] type[0] flag[5]
        H: 1280 1390 1430 1650
        V: 720 725 730 750
    Esmart0-win0: ACTIVE
```

```
        win_id: 0
        format: XR24 little-endian (0x34325258) pixel_blend_mode[0] glb_alpha[0xff]
        color: SDR[0] color-encoding[BT.601] color-range[Limited]
        rotate: xmirror: 0 ymirror: 0 rotate_90: 0 rotate_270: 0
        csc: y2r[0] r2y[0] csc mode[0]
        zpos: 0
        src: pos[0, 0] rect[1280 x 720]
        dst: pos[0, 0] rect[1280 x 720]
        buf[0]: addr: 0x0000000001017000 pitch: 5120 offset: 0
Video Port1: ACTIVE
    Connector:DP-5      Encoder: DP-MST 1
        bus_format[100a]: RGB888_1X24
        overlay_mode[0] output_mode[f] SDR[0] color-encoding[BT.709] color-range[Limited]
    Display mode: 1280x720p60
        clk[74250] real_clk[74250] type[0] flag[5]
        H: 1280 1390 1430 1650
        V: 720 725 730 750
    Esmart1-win0: ACTIVE
        win_id: 1
        format: XR24 little-endian (0x34325258) pixel_blend_mode[0] glb_alpha[0xff]
        color: SDR[0] color-encoding[BT.601] color-range[Limited]
        rotate: xmirror: 0 ymirror: 0 rotate_90: 0 rotate_270: 0
        csc: y2r[0] r2y[0] csc mode[0]
        zpos: 1
        src: pos[0, 0] rect[1280 x 720]
        dst: pos[0, 0] rect[1280 x 720]
        buf[0]: addr: 0x00000000001e1000 pitch: 5120 offset: 0
Video Port2: ACTIVE
    Connector:DP-6      Encoder: DP-MST 2
        bus_format[100a]: RGB888_1X24
        overlay_mode[0] output_mode[f] SDR[0] color-encoding[BT.709] color-range[Limited]
    Display mode: 1280x720p60
        clk[74250] real_clk[74250] type[0] flag[5]
        H: 1280 1390 1430 1650
        V: 720 725 730 750
    Esmart2-win0: ACTIVE
        win_id: 2
        format: XR24 little-endian (0x34325258) pixel_blend_mode[0] glb_alpha[0xff]
        color: SDR[0] color-encoding[BT.601] color-range[Limited]
        rotate: xmirror: 0 ymirror: 0 rotate_90: 0 rotate_270: 0
        csc: y2r[0] r2y[0] csc mode[0]
        zpos: 2
        src: pos[0, 0] rect[1280 x 720]
        dst: pos[0, 0] rect[1280 x 720]
        buf[0]: addr: 0x00000000015e2000 pitch: 5120 offset: 0
```

It can be seen that the summary has additional Encoder information.

On RK3576, one SST mode Encoder and three MST mode Encoders are registered. The correspondence between the 3 MST Encoders and the three display data streams of DP is as follows:

```
DP-MST 0 --> Stream-0
DP-MST 1 --> Stream-1
DP-MST 2 --> Stream-2
```

When the Encoder is an MST Encoder, it indicates that DP is operating in MST mode. If the Encoder corresponding to the DP Connector is TMDS-xxx, it indicates that DP is operating in SST mode. For example:

```
root@linaro-alip:/# cat /sys/kernel/debug/dri/0/summary
Video Port0: ACTIVE
    Connector:DP-1      Encoder: TMDS-184
        bus_format[1018]: RGB101010_1X30
        overlay_mode[0] output_mode[f] SDR[0] color-encoding[BT.709] color-range[Limited]
    Display mode: 1280x720p60
        clk[74250] real_clk[74250] type[0] flag[5]
        H: 1280 1390 1430 1650
        V: 720 725 730 750
    Esmart0-win0: ACTIVE
        win_id: 0
        format: XR24 little-endian (0x34325258) pixel_blend_mode[0] glb_alpha[0xff]
        color: SDR[0] color-encoding[BT.601] color-range[Limited]
        rotate: xmirror: 0 ymirror: 0 rotate_90: 0 rotate_270: 0
        csc: y2r[0] r2y[0] csc mode[0]
        zpos: 0
        src: pos[0, 0] rect[1280 x 720]
        dst: pos[0, 0] rect[1280 x 720]
        buf[0]: addr: 0x000000000090f000 pitch: 5120 offset: 0
Video Port1: DISABLED
Video Port2: DISABLED
```

3.7 Viewing Current Display Clock

Obtain the entire clock tree:

```
cat /sys/kernel/debug/clk/clk_summary
```

Obtain the dp aux 16M clk:

```
cat /sys/kernel/debug/clk/clk_summary | grep -e "clk_aux16m_
```

Obtain the vop dclk:

```
cat /sys/kernel/debug/clk/clk_summary | grep -e "dclk
```

3.8 Adjusting DRM Log Level

DRM has the following print level definitions, which can be dynamically enabled according to needs to print the corresponding logs:

```
enum drm_debug_category {
        /**
         * @DRM_UT_CORE: Used in the generic drm code: drm_ioctl.c, drm_mm.c,
         * drm_memory.c, ...
         */
        DRM_UT_CORE             = 0x01,
        /**
         * @DRM_UT_DRIVER: Used in the vendor specific part of the driver: i915,
         * radeon, ... macro.
         */
        DRM_UT_DRIVER           = 0x02,
        /**
         * @DRM_UT_KMS: Used in the modesetting code.
         */
        DRM_UT_KMS              = 0x04,
        /**
         * @DRM_UT_PRIME: Used in the prime code.
         */
        DRM_UT_PRIME            = 0x08,
        /**
         * @DRM_UT_ATOMIC: Used in the atomic code.
         */
        DRM_UT_ATOMIC           = 0x10,
        /**
         * @DRM_UT_VBL: Used for verbose debug message in the vblank code.
         */
        DRM_UT_VBL              = 0x20,
        /**
         * @DRM_UT_STATE: Used for verbose atomic state debugging.
         */
        DRM_UT_STATE            = 0x40,
        /**
         * @DRM_UT_LEASE: Used in the lease code.
         */
        DRM_UT_LEASE            = 0x80,
        /**
         * @DRM_UT_DP: Used in the DP code.
         */
        DRM_UT_DP               = 0x100,
       /**
         * @DRM_UT_DRMRES: Used in the drm managed resources code.
         */
        DRM_UT_DRMRES           = 0x200,
};
```

When troubleshooting DP interfaces, especially for commit anomalies, it is common to enable ATOMIC as follows:

```
echo 0x10 > /sys/module/drm/parameters/debug
```

If you need to print DPCD read/write logs, enter the following command:

```
echo 0x100 > /sys/module/drm/parameters/debug
```

3.9 Viewing DP MST Information

For DP interfaces that support MST, a SST Connector is registered by default, and its debugfs path is fixed. The MST Connector is dynamically registered and unregistered when devices are plugged in or out. Therefore, the node for viewing DP MST information is placed under the debugfs path of the SST Connector registered by the DP interface. For RK3576, the command is as follows:

```
root@linaro-alip:/# cat /sys/kernel/debug/dri/0/DP-1/dp_mst_info
        mstb - [000000003a17fc25]: num_ports: 4
        port 3 - [00000000099bbb63] (output - SST SINK): ddps: 1, ldps: 0, sdp: 1/1, fec:
false, conn: 00000000c74ff83b
        port 2 - [00000000505d66cf] (output - MST BRANCHING): ddps: 1, ldps: 0, sdp: 0/0,
fec: true, conn: 00000000b57a2623
                mstb - [0000000090afd0f3]: num_ports: 3
                port 1 - [00000000072aa867] (output - NONE): ddps: 0, ldps: 0, sdp: 0/0,
fec: false, conn: 000000005ba2b268
                port 8 - [0000000046d78f33] (output - SST SINK): ddps: 1, ldps: 0, sdp: 1/2,
fec: true, conn: 000000002668a7af
                port 0 - [00000000a70f650f] (input - NONE): ddps: 1, ldps: 0, sdp: 0/0, fec:
false, conn: 0000000000000000
        port 1 - [000000003fd2f64a] (output - SST SINK): ddps: 1, ldps: 0, sdp: 1/1, fec:
false, conn: 000000005ff7f122
        port 0 - [00000000c8a5769d] (input - NONE): ddps: 1, ldps: 0, sdp: 0/0, fec: false,
conn: 0000000000000000

*** Atomic state info ***
payload_mask: 7, max_payloads: 3, start_slot: 1, pbn_div: 60

| idx | port | vcpi | slots | pbn | dsc |     sink name     |
     1      1      1 06 - 10   266     N              U27U2D
     2      8      2 11 - 15   266     N        DELL U2723QE
     3      3      3 01 - 05   266     N             U28E590

*** DPCD Info ***
dpcd: 14 1e c4 81 01 11 01 83 2a 3f 04 00 00 00 84
faux/mst: 00 01
mst ctrl: 07
branch oui: 90cc24 devid: SYNAS revision: hw: 1.0 sw: 5.5
```

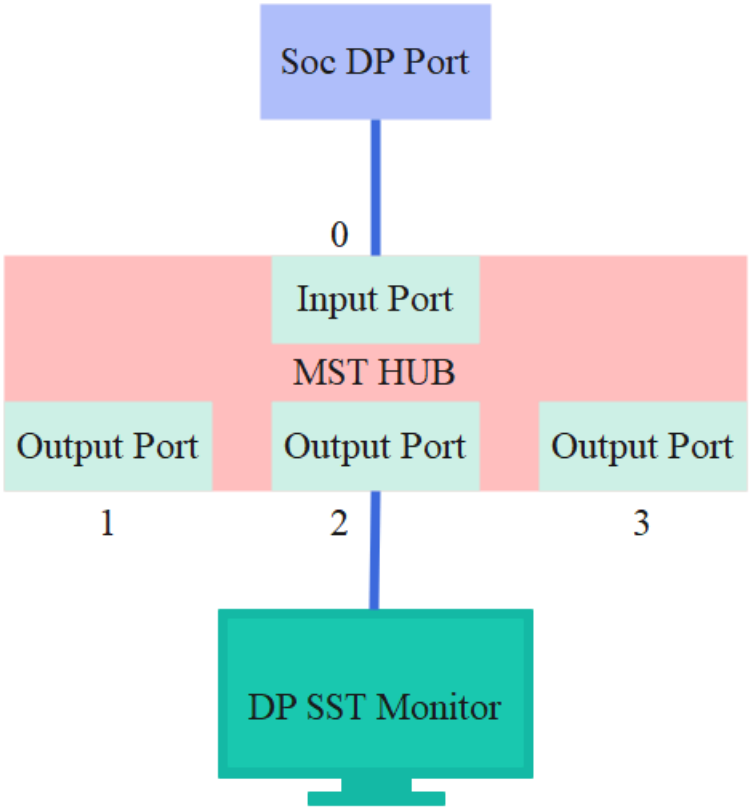
```
payload table: 03 03 03 03 03 03 01 01 01 01 01 02 02 02 02 02 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00

*** Connector path info ***
connector name | connector path
DP-2             mst:185-1
DP-3             mst:185-2
DP-6             mst:185-3
DP-5             mst:185-2-8
DP-7             mst:185-2-1
```

3.9.1 MST Port Info

The first part is the device connection topology structure.

```
        mstb - [000000003a17fc25]: num_ports: 4
        port 3 - [00000000099bbb63] (output - SST SINK): ddps: 1, ldps: 0, sdp: 1/1, fec:
false, conn: 00000000c74ff83b
        port 2 - [00000000505d66cf] (output - MST BRANCHING): ddps: 1, ldps: 0, sdp: 0/0,
fec: true, conn: 00000000b57a2623
                mstb - [0000000090afd0f3]: num_ports: 3
                port 1 - [00000000072aa867] (output - NONE): ddps: 0, ldps: 0, sdp: 0/0,
fec: false, conn: 000000005ba2b268
                port 8 - [0000000046d78f33] (output - SST SINK): ddps: 1, ldps: 0, sdp: 1/2,
fec: true, conn: 000000002668a7af
                port 0 - [00000000a70f650f] (input - NONE): ddps: 1, ldps: 0, sdp: 0/0, fec:
false, conn: 0000000000000000
        port 1 - [000000003fd2f64a] (output - SST SINK): ddps: 1, ldps: 0, sdp: 1/1, fec:
false, conn: 000000005ff7f122
        port 0 - [00000000c8a5769d] (input - NONE): ddps: 1, ldps: 0, sdp: 0/0, fec: false,
conn: 0000000000000000
```

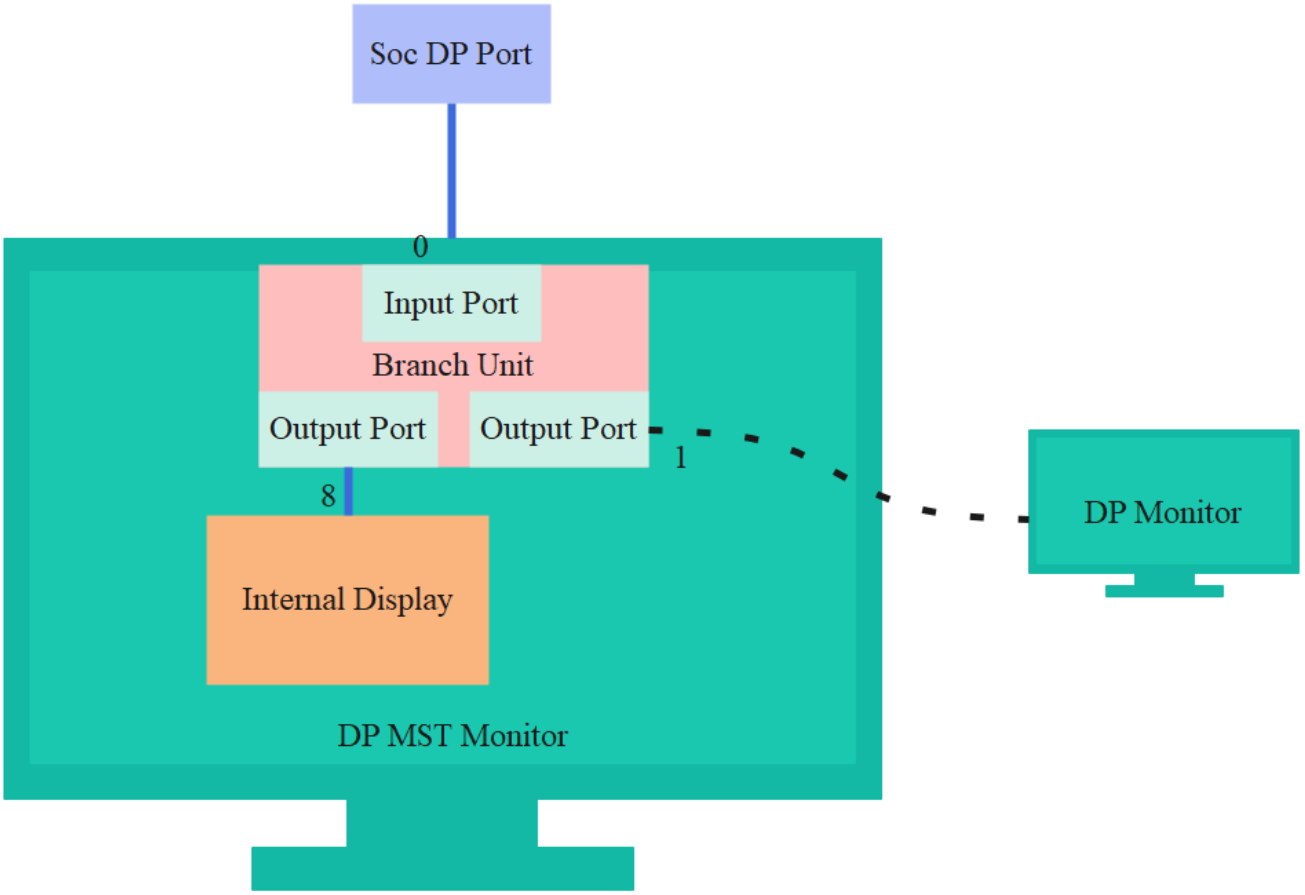
Recall Section 1.1.3, for RK3576 to operate in MST mode, it is necessary to connect an MST HUB or MST monitor. Each input and output port of the HUB or monitor has a unique number. The following shows the port numbers for a 1-input, 3-output MST HUB:



The above figure shows that the input port and output ports are physical interfaces that can be connected to other external devices. According to the DP protocol, the physical ports can use numbers 0 to 7, and the input port number must be smaller than the output port number. Generally, MST HUBs start numbering from 0. In the above MST HUB, there are a total of 4 ports, where Port 0 is the Input Port, and Port 1/2/3 are Output Ports. Port 2 is connected to an SST display, and the obtained topology structure information is as follows:

```
        mstb - [00000000c8269fca]: num_ports: 4
        port 3 - [0000000007690f7c] (output - NONE): ddps: 0, ldps: 0, sdp: 0/0, fec: false,
conn: 00000000d0584c6f
        port 2 - [00000000812cc57a] (output - SST SINK): ddps: 1, ldps: 0, sdp: 1/1, fec:
false, conn: 000000001ef4a070
        port 1 - [00000000efb170aa] (output - NONE): ddps: 0, ldps: 0, sdp: 0/0, fec: false,
conn: 000000001455081f
        port 0 - [000000005aa73543] (input - NONE): ddps: 1, ldps: 0, sdp: 0/0, fec: false,
conn: 0000000000000000
```

If an MST monitor is connected, an MST monitor generally has one DP input and one DP output, as shown in the figure below:

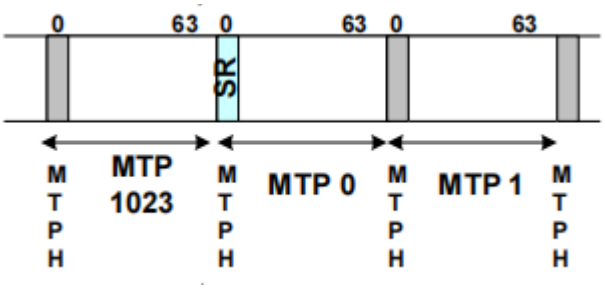


In the above figure, Port 0 is the Input Port, Port 1 is the Output Port, and Port 8 is the internal Output Port of the monitor. These internal ports of the MST monitor are logical ports, and the port numbers range from 8 to 15. When the above MST monitor is not daisy-chained to other monitors, the obtained topology structure information is as follows:

```
        mstb - [0000000019f71241]: num_ports: 3
        port 1 - [0000000095fb1fc0] (output - NONE): ddps: 0, ldps: 0, sdp: 0/0, fec: false,
conn: 000000004d03ffa2
        port 8 - [00000000a66da753] (output - SST SINK): ddps: 1, ldps: 0, sdp: 1/2, fec:
true, conn: 000000009a417589
        port 0 - [000000009a0122a8] (input - NONE): ddps: 1, ldps: 0, sdp: 0/0, fec: false,
conn: 0000000000000000
```

3.9.2 Atomic state info

In DP MST mode, the basic data packet format is called Multi-Stream Transport Packet (MTP). An MTP consists of 64 link symbols, also known as 64 time slots, numbered from 0 to 63. Among them, time slot 0 is for MTPH, and the other time slots can be used to transmit data for various display paths. The MTP format is shown below.



The following payload info table shows the port number of the MST device, the display path number (vcpi), the time slots occupied by the corresponding display path, the bandwidth (pbn) occupied by the display path, and the name of the sink (monitor).

```
| idx | port | vcpi | slots | pbn | dsc |     sink name     |
     1      1      1 06 - 10   266     N              U27U2D
     2      8      2 11 - 15   266     N        DELL U2723QE
     3      3      3 01 - 05   266     N             U28E590
```

3.9.3 DPCD Info

This section contains the obtained DPCD information, with a focus on the payload table, which records the detailed allocation of time slots.

```
*** DPCD Info ***
dpcd: 14 1e c4 81 01 11 01 83 2a 3f 04 00 00 00 84
faux/mst: 00 01
mst ctrl: 07
branch oui: 90cc24 devid: SYNAS revision: hw: 1.0 sw: 5.5
payload table: 03 03 03 03 03 03 01 01 01 01 01 02 02 02 02 02 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00
```

3.9.4 Connector Path Info

The connector name represents the dynamically registered DP connector, and the connector path represents the corresponding path.

```
*** Connector path info ***
connector name | connector path
DP-2             mst:185-1
DP-3             mst:185-2
DP-6             mst:185-3
DP-5             mst:185-2-8
DP-7             mst:185-2-1
```

4. FAQ

4.1 No Display or Display Abnormality after DP Insertion

First, check if the following log appears:

```
[   14.857002] rockchip-vop2 fdd90000.vop: [drm:vop2_crtc_atomic_enable] Update mode to
1920x1080p60, type: 10(if:200) for vp2 dclk: 148500000
[   14.857149] rockchip-vop2 fdd90000.vop: [drm:vop2_crtc_atomic_enable] dclk_out2 div: 2
dclk_core2 div: 2
[   14.857868] rockchip-vop2 fdd90000.vop: [drm:vop2_crtc_atomic_enable] set dclk_vop2 to
148500000, get 148500000
[   14.872406] dw-dp fde50000.dp: full-training link: 2 lanes at 5400 MHz
[   14.893269] dw-dp fde50000.dp: clock recovery succeeded
[   14.899797] dw-dp fde50000.dp: channel equalization succeeded
```

4.1.1 DP Link Training Successful

When the above log appears, it indicates that the DP connection has been detected, and the DP has successfully completed link training and is outputting an image. The reasons for no display or display abnormalities may be as follows:

1. Inaccurate dclk division

You can see the following log. The requested dclk is 25.175MHz, but the actual divided clock is 20MHz. If there is a clock division issue like this, capture the complete log and provide the clock tree log for further analysis.

```
[ 268.733803] rockchip-vop2 fdd90000.vop: [drm:vop2_crtc_atomic_disable] Crtc atomic disable
vp2
[ 268.759178] rockchip-vop2 fdd90000.vop: [drm:vop2_crtc_atomic_enable] Update mode to
640x480p60, type: 10(if:200) for vp2 dclk: 25175000
[ 268.759447] rockchip-vop2 fdd90000.vop: [drm:vop2_crtc_atomic_enable] dclk_out2 div: 2
dclk_core2 div: 2
[ 268.759665] rockchip_rk3588_pll_set_rate: Invalid rate : 25175000 for pll clk pll_v0pll
[ 268.759715] rockchip-vop2 fdd90000.vop: [drm:vop2_crtc_atomic_enable] set dclk_vop2 to
25175000, get 20000000
[ 268.775591] dw-dp fde50000.dp: full-training link: 4 lanes at 2700 MHz
[ 268.790059] dw-dp fde50000.dp: clock recovery succeeded
[ 268.795376] dw-dp fde50000.dp: channel equalization succeeded
```

1. No layer allocated

Userspace has not allocated a layer. Execute `cat /sys/kernel/debug/dri/0/summary`. If the obtained information is as follows, indicating no layer information, further analysis is needed from the userspace part.

```
rk3588_s:/ # cat /sys/kernel/debug/dri/0/summary
Video Port0: DISABLED
Video Port1: DISABLED
Video Port2: DISABLED
Video Port3: ACTIVE
    Connector: DSI-1
        bus_format[100a]: RGB888_1X24
        overlay_mode[0] output_mode[0] color_space[0], eotf:0
    Display mode: 1080x1920p60
        clk[132000] real_clk[132000] type[48] flag[a]
        H: 1080 1095 1099 1129
        V: 1920 1935 1937 1952
```

4.1.2 DP connected

If the log at the beginning of this section does not appear, first obtain the DP connection status as follows:

```
cat /sys/class/drm/card0-DP-1/status
```

If the DP is in a connected state, first analyze the log for any abnormal errors. If there are abnormal errors, analyze from the point of error. If the log shows no abnormalities, enable the DRM ATOMIC log level to reproduce the issue and confirm whether there is an abnormal return in the middle of drm atomic commit, which may indicate a failure in one of the atomic check stages.

4.1.3 DP disconnected

For standard DP port output, ensure that the HPD configuration is correct and the hardware connection is normal. For Type-C interfaces, refer to the subsequent analysis of Type-C interface connection abnormalities.

4.2 Type-C Interface Connection Abnormality

The Type-C interface connection abnormality here refers to exceptions occurring during the CC and PD phases. First, obtain the DP connection status as follows:

```
cat /sys/class/drm/card0-DP-1/status
```

When the connection is abnormal, the status obtained here will be disconnected.

Obtain the tcpm log through the tcpm debug node:

```
cat /sys/kernel/debug/usb/tcpm-2-0022
```

The log for a normal connection is as follows:

```
[   25.026952] AMS DISCOVER_IDENTITY start
```

```
[   25.026967] PD TX, header: 0x176f
[   25.035314] PD TX complete, status: 0
[   25.042866] PD RX, header: 0x524f [1]
[   25.042880] Rx VDM cmd 0xff00a041 type 1 cmd 1 len 5
[   25.042894] AMS DISCOVER_IDENTITY finished
[   25.042898] cc:=4
[   25.052343] Identity: 04e8:a020.0212
[   25.052364] AMS DISCOVER_SVIDS start
[   25.052372] PD TX, header: 0x196f
[   25.061314] PD TX complete, status: 0
[   25.067667] PD RX, header: 0x344f [1]
[   25.067680] Rx VDM cmd 0xff00a042 type 1 cmd 2 len 3
[   25.067695] AMS DISCOVER_SVIDS finished
[   25.067705] cc:=4
[   25.077097] SVID 1: 0xff01
[   25.077114] SVID 2: 0x4e8
[   25.077129] AMS DISCOVER_MODES start
[   25.077135] PD TX, header: 0x1b6f
[   25.086092] PD TX complete, status: 0
[   25.092224] PD RX, header: 0x264f [1]
[   25.092237] Rx VDM cmd 0xff01a043 type 1 cmd 3 len 2
[   25.092252] AMS DISCOVER_MODES finished
[   25.092256] cc:=4
[   25.101432]  Alternate mode 0: SVID 0xff01, VDO 1: 0x00000c05
[   25.101517] AMS DISCOVER_MODES start
[   25.101526] PD TX, header: 0x1d6f
[   25.109717] PD TX complete, status: 0
[   25.114919] PD RX, header: 0x284f [1]
[   25.114937] Rx VDM cmd 0x4e8a043 type 1 cmd 3 len 2
[   25.114951] AMS DISCOVER_MODES finished
[   25.114956] cc:=4
[   25.124604] AMS DFP_TO_UFP_ENTER_MODE start
[   25.124608] PD TX, header: 0x1f6f
[   25.134560] PD TX complete, status: 0
[   25.137903] PD RX, header: 0x1a4f [1]
[   25.137917] Rx VDM cmd 0xff01a144 type 1 cmd 4 len 1
[   25.137930] AMS DFP_TO_UFP_ENTER_MODE finished
[   25.137936] cc:=4
[   25.145828] AMS STRUCTURED_VDMS start
[   25.145836] PD TX, header: 0x216f
[   25.154942] PD TX complete, status: 0
[   25.161111] PD RX, header: 0x2c4f [1]
[   25.161125] Rx VDM cmd 0xff01a150 type 1 cmd 16 len 2 //STATUS UPDATE
[   25.161138] AMS STRUCTURED_VDMS finished
[   25.161142] cc:=4
[   25.171888] AMS STRUCTURED_VDMS start
[   25.171911] PD TX, header: 0x236f
[   25.182016] PD TX complete, status: 0
[   25.185550] PD RX, header: 0x1e4f [1]
[   25.185563] Rx VDM cmd 0xff01a151 type 1 cmd 17 len 1 //CONFIGURATION
[   25.185577] AMS STRUCTURED_VDMS finished
[   25.185581] cc:=4
[   26.392673] PD RX, header: 0x204f [1]
```

```
[   26.392687] Rx VDM cmd 0xff018106 type 0 cmd 6 len 2 //ATTENTION
```

From the log, it can be seen that the normal complete process includes interactions of commands such as DISCOVER_IDENTITY, DISCOVER_MODES, DFP_TO_UFP_ENTER_MODE, STATUS UPDATE, CONFIGURATION, and ATTENTION. If the above interaction process is abnormal, it indicates that the PD interaction has failed.

If the above process is abnormal, you can increase the I2C rate of the PD chip for further testing. If the problem still cannot be resolved, it is necessary to provide the complete tcpm log and the PD chip log for further analysis.

4.3 AUX_CH Abnormality

When AUX_CH is abnormal, it will cause exceptions in reading and writing DPCD and reading EDID. The log may show the following error:

```
[ 1368.952182] dw-dp fde50000.dp: failed to probe DP link: -110
```

If it is not certain, you can enable the following switch of the DRM debug log:

```
echo 0x100 > /sys/module/drm/parameters/debug
```

Obtain the DPCP read/write log through dmesg. The normal DPCP read/write log is as follows, with ret being 0:

```
[ 6329.554538] rockchip-drm display-subsystem: [drm:drm_dp_dpcd_probe] fde50000.dp: 0x00000
AUX -> (ret=  1) 12
[ 6329.554939] rockchip-drm display-subsystem: [drm:drm_dp_dpcd_read] fde50000.dp: 0x00000
AUX -> (ret= 15) 12 14 c2 81 01 01 01 81 02 02 06 00 00 00 81
[ 6329.555383] rockchip-drm display-subsystem: [drm:drm_dp_dpcd_probe] fde50000.dp: 0x00000
AUX -> (ret=  1) 12
```

When AUX_CH is abnormal, the ret value is an exception type value, as follows:

```
[   31.116976] rockchip-drm display-subsystem: [drm:drm_dp_dpcd_probe] fde50000.dp: 0x00000
AUX -> (ret=-110)
```

Possible causes of AUX_CH abnormality are as follows.

4.3.1 aux16m clk value abnormal

The aux16m clk rate is abnormal. The default value of aux16m clk is as follows. The parent clk of aux16m clk is GPLL, which is set to 1188MHz by default.

```
root@RK3588:/# cat /sys/kernel/debug/clk/clk_summary | grep "aux16m"
          clk_aux16m_1                1        2        0    15840000          0     0
50000
          clk_aux16m_0                1        2        0    15840000          0     0
50000
```

If the obtained GPLL is not 1188MHz and the aux16m clk is not the default value, first check whether there have been modifications to the files in `/drivers/clk/rockchip/` based on the SDK, or whether the parent of the VOP DCLK has been reconfigured.

4.3.2 phy power on/off process abnormal

This abnormality generally occurs in Type-C interfaces where USB and Type-C share the phy. If one side of the Type-C interface can work normally, but the other side reports an error when inserted, it may be that the USB phy did not exit properly when pulled out, causing the PHY to not be reinitialized when reinserted. You can add logs to first confirm whether phy power on/off is executed during USB plug-in and plug-out, and whether the PHY is reinitialized when the other side of the Type-C interface is reinserted. The log addition can refer to the following patch.

```
diff --git a/drivers/phy/rockchip/phy-rockchip-usbdp.c b/drivers/phy/rockchip/phy-rockchip-
usbdp.c
index c6fc4a2aa558..6b35e12f40aa 100644
--- a/drivers/phy/rockchip/phy-rockchip-usbdp.c
+++ b/drivers/phy/rockchip/phy-rockchip-usbdp.c
@@ -823,6 +823,7 @@ static int udphy_power_on(struct rockchip_udphy *udphy, u8 mode)
 {
        int ret;

+       dev_info(udphy->dev, "%s status:%x, mode:%x\n", __func__, udphy->status, mode);
        if (!(udphy->mode & mode)) {
                dev_info(udphy->dev, "mode 0x%02x is not support\n", mode);
                return 0;
@@ -859,6 +860,7 @@ static int udphy_power_off(struct rockchip_udphy *udphy, u8 mode)
 {
        int ret;

+       dev_info(udphy->dev, "%s status:%x, mode:%x\n", __func__, udphy->status, mode);
        if (!(udphy->mode & mode)) {
                dev_info(udphy->dev, "mode 0x%02x is not support\n", mode);
                return 0;
@@ -883,6 +885,7 @@ static int rockchip_dp_phy_power_on(struct phy *phy)
        struct rockchip_udphy *udphy = phy_get_drvdata(phy);
        int ret, dp_lanes;

+       dev_info(udphy->dev, "%s\n", __func__);
        mutex_lock(&udphy->mutex);

        dp_lanes = udphy_dplane_get(udphy);
@@ -914,6 +917,7 @@ static int rockchip_dp_phy_power_off(struct phy *phy)
        struct rockchip_udphy *udphy = phy_get_drvdata(phy);
```

```
        int ret;

+       dev_info(udphy->dev, "%s\n", __func__);
        mutex_lock(&udphy->mutex);
        ret = udphy_dplane_enable(udphy, 0);
        if (ret)
@@ -1028,6 +1032,7 @@ static int rockchip_u3phy_init(struct phy *phy)
        struct rockchip_udphy *udphy = phy_get_drvdata(phy);
        int ret = 0;

+       dev_info(udphy->dev, "%s\n", __func__);
        mutex_lock(&udphy->mutex);
        /* DP only or high-speed, disable U3 port */
        if (!(udphy->mode & UDPHY_MODE_USB) || udphy->hs) {
@@ -1047,6 +1052,7 @@ static int rockchip_u3phy_exit(struct phy *phy)
        struct rockchip_udphy *udphy = phy_get_drvdata(phy);
        int ret = 0;

+       dev_info(udphy->dev, "%s\n", __func__);
        mutex_lock(&udphy->mutex);
        /* DP only or high-speed */
        if (!(udphy->mode & UDPHY_MODE_USB) || udphy->hs)
@@ -1363,6 +1369,7 @@ static int rk3588_udphy_init(struct rockchip_udphy *udphy)
        const struct rockchip_udphy_cfg *cfg = udphy->cfgs;
        int ret;

+       dev_info(udphy->dev, "%s\n", __func__);
        /* enable rx lfps for usb */
        if (udphy->mode & UDPHY_MODE_USB)
                grfreg_write(udphy->udphygrf, &cfg->grfcfg.rx_lfps, true);
```

4.3.3 DP dual mode cable causing abnormality

DP dual mode requires that the DP port supports both DP signal output and HDMI TMDS signal transmission. The AUX channel must also support both DP AUX and DDC (I2C).

RK3588 DP does not support DP dual mode. If a cable that supports DP dual mode is connected, it will cause AUX_CH abnormality. This generally occurs with DP to HDMI cables or DP to HDMI converters. If a DP to HDMI cable is used to connect an HDMI monitor and an AUX_CH abnormality occurs, and the cable supports HDMI 2.0 or lower protocol versions, it is likely that the cable supports DP dual mode. It is recommended to replace it with a cable that supports HDMI 2.0 or higher protocol versions.

4.3.4 Signal interference causing abnormality

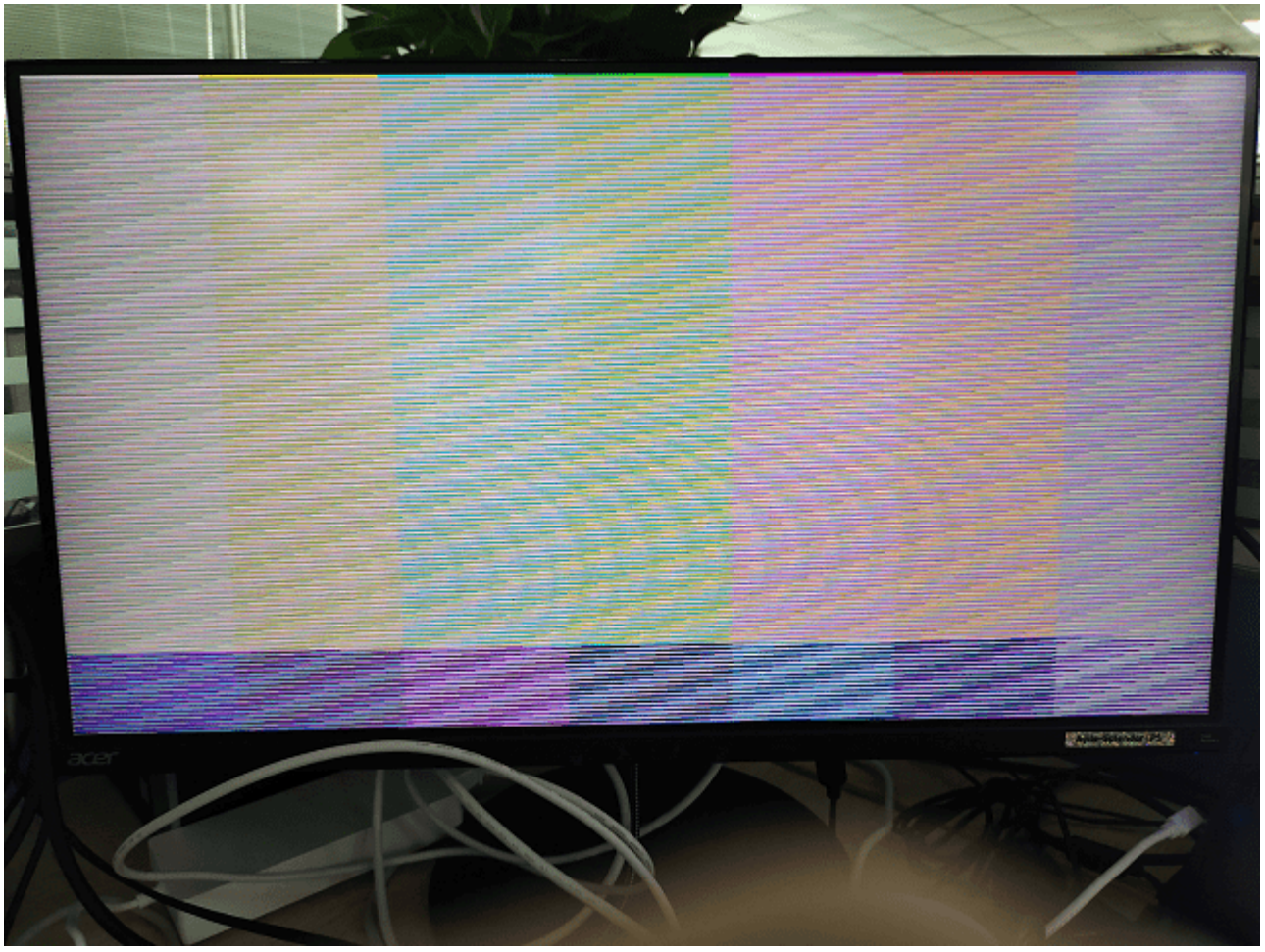
This issue generally occurs in scenarios where Type-C is directly connected. The DP AUX is interfered with by signals on USB DP/DM. You can confirm this by disconnecting USB DP/DM on the hardware and observing whether the problem reproduces. To solve this issue, one way is to use a higher quality Type-C with better shielding between signals. Another way is to avoid using USB DP/DM for data transmission.

4.3.5 Hardware abnormality

First, it is necessary to confirm whether the hardware connection path is normal and whether the AUX differential signal is transmitted normally. Check for any soldering issues. Secondly, ensure that the peripheral circuit of the AUX_CH path is designed in accordance with the DP protocol. If a conversion chip is connected, first confirm whether the peripheral circuit of the conversion chip is normal.

4.4 4K 120Hz Output Configuration

The default VOP ACLK of RK3588 is 500M. For high pixel clk configurations such as 4K 120Hz output, performance issues may lead to display abnormalities as shown below:



For such issues, the VOP ACLK needs to be increased to 800M:

```
&vop {
    assigned-clocks = <&cru ACLK_VOP>;
    assigned-clock-rates = <800000000>;
};
```

Obtain VOP ACLK as follows:

```
cat /sys/kernel/debug/clk/clk_summary | grep "aclk_vop"
```

4.5 DP Bandwidth Calculation

4.5.1 SST Mode Bandwidth Calculation

To obtain the bandwidth supported by each DP lane, use the following formula:

$$bandwidth_per_lane = lane_count * pixel_clk * bit_per_pixel * 1.25$$

Here, `bit_per_pixel` represents the number of bits per pixel, `1.25` is the encoding conversion efficiency of the phy lane, and `lane_count` is the number of available lanes. The resulting `bandwidth_per_lane` is the minimum bandwidth that each lane needs to provide. If the current lane rate is smaller than the required minimum bandwidth, the corresponding pixel clk display mode will be filtered out by the DP driver.

When using a conversion cable or docking station, it is necessary to ensure that the supported lane rate and lane count of the cable or HUB meet the bandwidth requirements. If not, it is necessary to replace the cable or HUB with one that supports a higher lane rate and more lanes.

For example, for a HUB with 2 lanes and a maximum lane rate of 5.4 Gbps/lane, if the output is 4K@60Hz with a pixel clock of 594MHz and RGB888 format data, the required bandwidth per lane is:
bandwidth_per_lane=2594×24×1.25=8.91 Gbps/lane>5.4 Gbps/lane

It can be seen that the current HUB does not support the output of 4K@60Hz with a pixel clock of 594MHz and RGB888 format data. It is necessary to use a 4-lane output HUB to increase the PHY lane bandwidth, or output YUV420 format data to reduce the required PHY lane bandwidth.

4.5.2 MST Mode Bandwidth Calculation

The bandwidth calculation in MST mode is similar to that in SST mode. The formula for calculating the bandwidth required by each stream per lane is the same as in SST mode. However, it is necessary to consider whether the total bandwidth will exceed the limit when multiple streams are output simultaneously. Additionally, in MST mode, the MTPH packet header also occupies a certain amount of bandwidth, which needs to be considered for corresponding bandwidth loss.

For example, calculating based on the maximum bandwidth of RK3576, which is 4 lanes and 8.1 Gbps:

One MTP has 64 time slots, of which one time slot is for MTPH. Therefore, the maximum bandwidth supported by each lane is: bandwidth_per_lane_max=8.1×6463=7.97 Gbps

For a display output of 4096x2160@60Hz with a pixel clock of 594MHz and RGB888 format, the bandwidth occupied per lane is bandwidth_per_lane=4594×24×1.25=4.46 Gbps

For a display output of 2560x1440@60Hz with a pixel clock of 297MHz and RGB888 format, the bandwidth occupied per lane is bandwidth_per_lane=4297×24×1.25=2.23 Gbps

For a display output of 1920x1080@60Hz with a pixel clock of 148.5MHz and RGB888 format, the bandwidth occupied per lane is bandwidth_per_lane=4148.5×24×1.25=1.12 Gbps

For RK3576, with the maximum bandwidth of 4 lanes and 8.1 Gbps, the maximum capability for three streams output simultaneously is as follows. The resolutions are set as 4096x2160@60Hz, 2560x1440@60Hz, and 1920x1080@60Hz for the three streams respectively. The total bandwidth consumed per lane is less than the maximum supported total bandwidth per lane: bandwidth_per_lane_total=4.46+2.23+1.12=7.81 Gbps<7.97

Gbps

This is also the maximum capability supported by RK3576 in MST mode for three streams output.

4.6 DP Timing Limitations

For non-standard resolutions, if there is a scenario where the porch is too small, it may lead to DP being unable to output a display. Currently, the DP driver limits the minimum value of HBP to 16 and the minimum value of HSYNC to 9. If the values are lower than the minimum, the driver will adjust the corresponding HBP or HSYNC to the supported minimum value.

4.7 MST Mode Usage Limitations

4.7.1 Capability Limitations

For the DP interface of RK3576, the maximum capability of each stream is as follows:

DP Stream Channel	max width	max height	max pixel clock
Stream-0	4096	2160	1188MHz
Stream-1	2560	1440	300MHz
Stream-2	1920	1080	150MHz

For the VOP of RK3576, the maximum output capability of each Video Port is as follows:

Vop Video Port	max width	max height	max pixel clock
Video Port 0	4096	2160	1200MHz
Video Port 1	2560	1600	300MHz
Video Port 2	1920	1080	150MHz

Considering the output capabilities of VOP and DP, if you want to use DP MST on RK3576 for three-screen extended display, the maximum resolution that can be output is recommended to be configured as follows: Video Port 0 -> DP Stream-0, Video Port 1 -> DP Stream-1, Video Port 2 -> DP Stream-2. The DTS configuration is as follows:

```
&dp0 {
    status = "okay";
};

&dp0_in_vp0 {
    status = "okay";
};
```

```
&dp0_in_vp1 {
    status = "disabled";
};

&dp0_in_vp2 {
    status = "disabled";
};

&dp1 {
    status = "okay";
};

&dp1_in_vp0 {
    status = "disabled";
};

&dp1_in_vp1 {
    status = "okay";
};

&dp1_in_vp2 {
    status = "disabled";
};

&dp2 {
    status = "okay";
};

&dp2_in_vp0 {
    status = "disabled";
};

&dp2_in_vp1 {
    status = "disabled";
};

&dp2_in_vp2 {
    status = "okay";
};
```

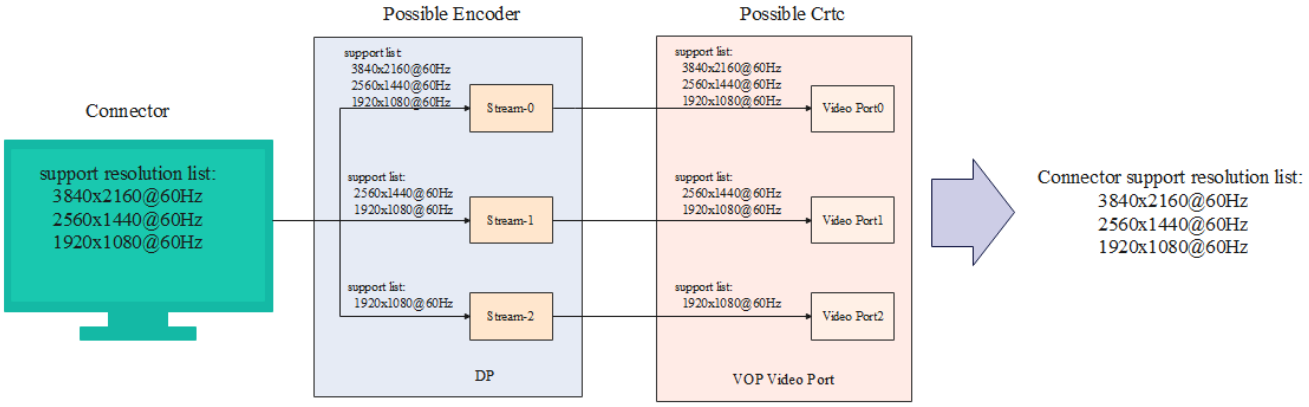
If you want to use DP MST on RK3576 for three-screen mirrored display, the maximum resolution that can be output is 1920x1080@60Hz. It is recommended to configure Video Port 2 -> DP Stream-0, Video Port 2 -> DP Stream-1, Video Port 2 -> DP Stream-2. The DTS configuration is as follows:

```
&dp0 {
    status = "okay";
};

&dp0_in_vp0 {
    status = "disabled";
};
```

```
&dp0_in_vp1 {
    status = "disabled";
};

&dp0_in_vp2 {
    status = "okay";
};

&dp1 {
    status = "okay";
};

&dp1_in_vp0 {
    status = "disabled";
};

&dp1_in_vp1 {
    status = "disabled";
};

&dp1_in_vp2 {
    status = "okay";
};

&dp2 {
    status = "okay";
};

&dp2_in_vp0 {
    status = "disabled";
};

&dp2_in_vp1 {
    status = "disabled";
};

&dp2_in_vp2 {
    status = "okay";
};
```

4.7.2 Resolution Filtering

In Linux DRM framework, DP MST dynamically registers Connectors. When a Connector is connected, it is not possible to determine which Encoder the Connector will ultimately use for output. For example, in the 3-screen extended display configuration mentioned in the previous section, only the resolutions that are not supported by all three DP Streams will be filtered out, as shown in the figure below:



However, due to the differences in the output capabilities of each DP Stream and each VOP Video Port, some resolutions may not be able to be output on certain display paths. For example, if you want to output 3840x2160@60Hz on the display path of Video Port 2 -> DP Stream-2, it will not display properly.

For such cases, it is necessary for the user-space application to limit the resolution output for specific display paths. Alternatively, ensure that the maximum resolution of the connected display device does not exceed the maximum supported resolution of DP Stream-2.